SharkTooth 導讀教材

Oct 9, 2026 · @LIN YU HUNG

對照論文：SharkTooth: A Scalable Real-Time Algorithm for BLE-Based Wireless Body Sensor Network Synchronization（IEEE Internet of Things Journal, 12(22), pp. 46174–46192, 2025）。標記：【原文】論文明寫｜【補充】背景或推導｜【推測】論文沒寫、依據另附。行內數學以純文字符號呈現，獨立公式以 LaTeX 區塊呈現。

第 0 部分：閱讀地圖

0.1 三句話

1. **問題**：多個穿戴式感測節點以藍牙低功耗 (Bluetooth Low Energy, BLE) 的連線模式 (connection mode) 連到同一台電腦時，每個節點用自己的晶振計時，加上 BLE 傳輸延遲不可預測，各節點的資料流在時間上對不齊。
2. **重要性**：多肌肉 sEMG、sEMG + IMU 融合要求對齊誤差在一個取樣週期內；既有方法要嘛需要 MAC 層或硬體時間戳（商用 BLE 堆疊拿不到），要嘛只能用廣播模式 (advertising mode，單向、不能 read/write/notify)，要嘛只驗證 2–4 個節點或只能離線。
3. **方法**：只用應用層的兩個時間戳——節點本地時間 t_P（寫在封包裡）與 PC 收到時間 t_C——把差值 δT = t_C − t_P 建模成「線性漂移 + 不可預測延遲」，先用自適應的「下包絡追蹤」濾掉重傳與軟體造成的延遲尖峰得到 δT*，再用遞迴最小平方法 (Ordinary Least Squares, OLS) 估計漂移 α 與偏移 β，即時算出同步時間 t̂_S。

0.2 論文宣稱的貢獻（Abstract、§I-B p.46176、§VI 結論條列 p.46190）

- 【原文】完全在應用層運作，不需修改韌體、不需特殊硬體，相容商用現成 (Commercial Off-The-Shelf, COTS) BLE 裝置。【補充】「不改韌體」指的是 BLE 協定堆疊與 central dongle 的韌體；節點端的應用程式仍要自己把 t_P 與封包編號寫進封包。
- 【原文】自適應時間戳修正，同時處理確定性的時鐘漂移，以及封包重傳與協定變異造成的不可預測延遲。
- 【原文】可擴展性：最多 12 個節點、18 種網路配置（節點數 2/4/6/8/10/12 × 三種封包長度），每種配置做 20 次、每次 1 小時的擷取。
- 【原文】中位數「絕對平均同步誤差」在 0.21–0.62 ms 之間，即使在擁塞的網路情境。
- 【原文 Table II】這個範圍是第 6 個 600 s 區段的中位數：A2（2 節點、17 B）= 0.21 ms，C12（12 節點、244 B）= 0.62 ms；第 1 段則是 0.30–1.27 ms。第 95 百分位數則從 3.56 ms 降到 1.28 ms。報告時要附上這個條件；而且這個指標每段只取「最差的那一對節點」，屬於保守寫法。
- 【原文】即時（線上）運作，對比只能離線處理的方法。

- 【原文】三項分析：延遲變異與誤差的關係 (§V-A)、與既有方法比較 (§V-B)、誤差與網路吞吐量的關係，可當系統設計工具 (§V-C)。

0.3 方法流程圖

```
[感測節點 n]（驗證時共 12 個，每 4 個一組）
  ADC data-ready 中斷 → 立刻打本地時間戳 → 存入環形緩衝
  每 100 ms 打包：感測資料 + t_P(最後一筆樣本, 4 B) + packet ID(1 B)
        |  BLE notification（connection mode, CI = 30 ms, 遺失會重傳）
        ▼
[nRF52840 dongle = central, 原廠韌體] × 3
        |  USB serial
        ▼
[PC 上的 Python 控制程式]
  serial callback 觸發 → 立刻打 t_C
  對每個節點平行跑 SharkTooth：
   (1) δT = t_C − t_P  ；δ²T = 相鄰差  ；滑動標準差 σ_M      Eq.(3)(5)(7)
   (2) 不可預測延遲修正 → δT*（貼著 δT 下緣的直線）         Alg.2, Eq.(8)–(14)
   (3) 遞迴 OLS：δT* ≈ α·t_P + β（換新 pivot 後等 30 s）   Eq.(15)–(18)
   (4) t̂_S = t_P + α·t_P + β                                  Eq.(19)(20)
  輸出：每個封包的同步時間戳 t̂_S

[驗證]
  示波器 → 兩路反相 5 Hz 正弦 → 同時餵給所有節點（50 Hz 取樣）
  估計的節點間延遲 τ^（Eq.23, 24）  vs.  互相關求出的真實延遲 τ（Eq.25）
  → RSE = τ^ − τ（Eq.26）→ 每 600 s 取最差的一對（Eq.27）→ 平均|RSE|、std、P95
```

0.4 建議閱讀順序

層級	段落	理由
精讀	§III-A 時間戳模型（p.46178–46179，Eq.1–7，Fig.3、4）	整篇的地基：δT 由哪三種成分組成
精讀	§III-C 演算法（p.46180–46182，Alg.1、2，Eq.8–22）	核心貢獻
精讀	§IV-B 評估方法（p.46183–46185，Eq.23–27）	決定 0.21–0.62 ms 是怎麼量出來的
精讀	§V 開頭與 Fig.11 的趨勢描述（p.46185–46187）	主要結果
精讀（對你的專案）	§II-B 與 §IV-A 的連線間隔與吞吐量預算推理（p.46177、46183）	自製無線節點時直接用得到

層級	段落	理由
略讀	§I-A 相關研究（p.46175–46176）	記住分類即可：協定無關型 → BLE 廣播型 → BLE 連線型
略讀	§V-B 與既有方法比較（p.46187–46189）	需要 Table III 數值才有意義
先跳過	§V-A 的回歸細節（Fig.12）、§V-C 吞吐量（Fig.13）、作者簡介	設計系統時再回來

0.5 在你專案裡的位置

- 你下一步要做 Trigno 式架構：每條肌肉一個獨立無線節點加一個 base station，並預期跨節點時間同步是瓶頸。SharkTooth 的系統架構幾乎一樣（獨立 BLE 節點 → dongle → PC），而且只要求節點在封包放本地時間戳。
- 【推測】你目前的路徑 1 用 250 ms RMS 視窗、25 ms 步長。在這種設定下，節點間差幾毫秒對 RMS 包絡的影響可以忽略，sub-ms 精度可能高於你的需求；真正的風險更可能是掉包與吞吐量。第 6 部分會再展開。
- 【原文】論文未提供程式碼連結。第 7 部分的實作是依論文公式自行寫成，已在工作區執行驗證。
- 【更正】我在對話中說過 Table I–III 與 Fig.2–6、10–13 讀不到數值。改用頁面影像重讀後，這些表與圖都看得到，第 4 部分逐一解讀；從圖上目測的數值會標「約」。

第 1 部分：先備背景知識

以下由最基礎的概念往上疊。每個概念依序給：直覺、正式定義、小例子、本文用途、常見誤解。除非標【原文】，本部分皆為【補充】。

1.1 晶體振盪器、頻率容差 (ppm)、偏移與漂移

直覺：兩支手錶同時對時，一支每天快 2 秒。對時那一刻的差叫「偏移」，之後差距越拉越大的速度叫「漂移」。

定義：理想時間記為 t，節點的本地時鐘讀數 C(t) 可近似為

```
C(t) = (1+\rho)\,t + \theta
```

ρ 是相對頻率誤差（無單位，常以 ppm = 10^{-6} 表示），θ 是開機時刻不同造成的偏移。晶振規格寫的「±20 ppm」是頻率容差：實際頻率 f 與標稱頻率 f_0 的相對差 $|f - f_0|/f_0 \le 20 \times 10^{-6}$。溫度、電源、機械應力與老化會讓 ρ 慢慢改變【原文 §I p.46174】。

例子：【原文 §II-A】節點用 32.768 kHz 外部晶振，容差 ±20 ppm。一個 tick = 1/32768 s ≈ 30.5 µs。最壞情況下一小時累積的誤差：

$$3600\ \text{s} \times 20\times10^{-6} = 0.072\ \text{s} = 72\ \text{ms}$$

兩個節點一快一慢時，彼此差距最多 144 ms，遠大於 sEMG 的 1 ms 取樣週期。所以只對一次時是不夠的，必須持續估計漂移。

本文用途：Eq.(1) 的 α 就是漂移、β 就是偏移；Eq.(14) 的步長 10^{-6} 刻意取在漂移的數量級【原文 p.46181】。

常見誤解：「±20 ppm」不是實際漂移量，而是上界；實際 ρ 可能只有幾 ppm，而且會隨溫度變。另外，偏移可以一次修正，漂移則會一直累積。

1.2 時間同步的基本模型與三種架構

直覺：同步就是找一個換算公式，把每個節點的本地時間換成大家共用的時間軸。

定義：最常見的是兩參數仿射模型

$$t_S = (1+\alpha)\,t_P + \beta$$

要估 α、β，需要「同一瞬間」在兩個時鐘下的讀數配對，也就是 FTSP 所稱的參考點 (reference point)：一組 (全域時間, 本地時間)。三種典型架構：

架構	代表	做法	特點
接收端對接收端 (receiver–receiver)	RBS	參考節點廣播，接收者互相比較收到時間	消除發送端不確定性
雙向交換 (sender–receiver, two-way)	TPSN、NTP	一來一回四個時間戳	可同時估偏移與往返延遲
單向 (one-way) + 回歸	FTSP、SharkTooth	只有一個方向的時間戳配對，用回歸估 α、β	簡單，但無法單獨分辨「偏移」與「最小傳輸延遲」

例子（雙向公式，用來對比）：A 在 t_1 = 100 ms 送出，B 在自己的時鐘 t_2 = 150 ms 收到，B 在 t_3 = 152 ms 回覆，A 在 t_4 = 112 ms 收到。若兩向延遲相等：

$$\text{offset} = \frac{(t_2-t_1)+(t_3-t_4)}{2} = \frac{50+40}{2} = 45\ \text{ms},\qquad \text{delay} = \frac{(t_4-t_1)-(t_3-t_2)}{2} = \frac{12-2}{2}=5\ \text{ms}$$

本文用途：SharkTooth 是單向架構，只有節點 → PC 方向。【原文 §I-A】作者提到 BATS 的思路：節點只負責打時間戳，所有計算交給中心；SharkTooth 也是這種不對稱分工。

常見誤解：單向法估出來的 β 其實是「偏移 + 最小延遲」。但評估的是節點之間的相對誤差，所以所有節點共有的固定延遲會互相抵消；不共有的部分則不會（第 6 部分討論）。

1.3 BLE 的兩種通訊模式、GATT 與 notification

直覺：廣播模式像電台，大家聽、不能回話；連線模式像排班表固定的對講機，每隔固定時間輪到你講一次。

定義【原文 §I p.46174–46175】：

- 廣播 (advertising)：快速、單向、可一對多，但沒有安全性，只能傳簡單輸出。
- 連線 (connection)：一個中心 (central) 和多個周邊 (peripheral) 組成 piconet，雙向、安全。資料只在固定間隔的連線事件 (connection event) 交換，間隔稱為連線間隔 (Connection Interval, CI)，以分時避免碰撞。
- 通用屬性設定檔 (Generic Attribute Profile, GATT)：資料以 service → characteristic 組織，透過 read、write、notify 存取。notify 是周邊主動推資料給中心，用於即時串流。
- 連線模式有鏈路層確認 (acknowledgment) 與重傳：封包沒收到就在下一個連線事件重送。

例子：CI = 30 ms。某封包在上一個連線事件後 12 ms 才準備好，就要等 18 ms 才送得出去；若這次又丟包，再加 30 ms，總延遲變成 48 ms 再加上 USB 與軟體處理時間。

本文用途：Eq.(2)、(4) 的不可預測延遲 ε 主要就來自「等下一個連線事件」與「重傳」【原文 §III-A p.46178】。

常見誤解：「連線模式可靠，所以延遲固定」是錯的。可靠是用重傳換來的，代價正是延遲會跳 CI 的整數倍。

1.4 連線間隔、封包長度與吞吐量預算

直覺：一個 dongle 一個 CI 內要輪流服務所有節點；節點越多、每個節點講越久，CI 就得越長，每個節點能送的資料量就越少。

定義【原文 §IV-A p.46183】：送最大 244 B 的封包需要 7.5 ms 的連線事件長度；同一個 central 下 N 個節點，CI 至少約 N × 7.5 ms。為了讓遺失的封包有機會在下一個事件重傳，封包送出週期至少要 2 × CI。

例子（論文原數字，逐步算）：

- 12 個節點接 1 支 dongle：CI = 12 × 7.5 = 90 ms，送出週期 ≥ 180 ms，每節點吞吐量 ≤ 244 B / 0.18 s ≈ 1.355 kB/s。
- 一個 1 kHz、16 bit 的單通道 EMG 就要 1000 × 2 B = 2 kB/s，一個 208 Hz × 6 通道 × 16 bit 的 IMU 要 208 × 6 × 2 ≈ 2.5 kB/s，都超過 1.355 kB/s。
- 改成 3 支 dongle、各 4 個節點：CI = 4 × 7.5 = 30 ms，送出週期 ≥ 60 ms，每節點 ≤ 244 / 0.06 ≈ 4.066 kB/s，足夠。

【補充】244 B 這個數字來自 BLE 4.2 的資料長度擴展 (Data Length Extension)：鏈路層酬載最大 251 B，扣掉 L2CAP 標頭 4 B 與 ATT 標頭 3 B，剩 244 B。

本文用途：解釋為什麼驗證時固定 CI = 30 ms、用 3 支 dongle。

常見誤解：多加 dongle 並不會把問題簡化成「三個獨立的小網路」。【原文】PC 仍要同時處理三條資料流，而且三個網路在相近空間共用 2.4 GHz 頻段，碰撞機率增加。

1.5 時間戳打在哪一層：MAC 層 vs 應用層

直覺：越靠近天線打時間戳，越不受軟體排程干擾；越靠近應用程式，延遲雜訊越多，但越容易實作。

定義：一個封包從產生到被應用程式處理，會經過發送 (send)、存取通道 (access)、傳輸 (transmit)、傳播、接收 (receive) 等階段，每一段都可能帶延遲【原文 §I 引用 FTSP 等文獻】。MAC 層時間戳在「位元真正上空中」那一刻打，可以消除前後的軟體延遲；應用層時間戳則包含全部。

例子：本文 t_C 打在 PC 端 Python 的 serial callback 裡，所以包含：等連線事件 + 空中傳輸 + dongle 處理 + USB serial + 作業系統排程 + Python 執行緒排程。

本文用途：【原文 §I-A】商用 BLE 平台的協定堆疊與韌體架構通常不讓使用者拿到 MAC 層時間戳，所以 RBS/TPSN/FTSP 這類方法無法直接套用；SharkTooth 選擇接受應用層的雜訊，再用演算法把它濾掉。

常見誤解：「應用層時間戳就不能做到 sub-ms」——本文的重點正是用統計方法在應用層做到 sub-ms。

1.6 單向延遲的「最小值」原理（下包絡）

直覺：延遲只會被拉長、不會被縮短。所以觀測到的延遲裡，最小的那幾個點最接近「沒受干擾」的真實趨勢。

定義：把延遲寫成固定最小值加上非負的額外量：

```
\delta T_i = \alpha\, t_{P,i} + \beta + d_{\min} + e_i,\qquad e_i \ge 0
```

如果 e_i 偶爾會非常大（重傳、程式卡住），用全部點做平均回歸會被拉高；貼著資料「下緣」畫線，則只受 e_i 最小值影響。

例子：真實趨勢 0、5 個觀測的額外延遲 e = (0, 30, 2, 0, 31) ms。平均是 12.6 ms，最小值是 0 ms。若尖峰集中在後段，平均回歸還會把斜率拉歪，下緣則不會。

本文用途：Algorithm 2 的 δT* 就是一條「貼著 δT 下緣的直線」【原文 §III-C p.46180】。

常見誤解：下緣不等於「去掉離群值後的平均」。下緣估的是 β + d_min，平均估的是 β + d_min + E[e]；前者對尖峰穩健，後者不是。

1.7 最小平方回歸、正規方程與 2×2 反矩陣

直覺：找一條直線，讓所有點到直線的垂直距離平方和最小。

定義：模型 y_i = α x_i + β + 雜訊。最小化

$$J(\alpha,\beta) = \sum_{i=1}^{n} (y_i - \alpha x_i - \beta)^2$$

對 α、β 偏微分設為 0，得到正規方程 (normal equations)：

$$\begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{bmatrix}\begin{bmatrix}\alpha\\ \beta\end{bmatrix} = \begin{bmatrix}\sum x_i y_i\\ \sum y_i\end{bmatrix}$$

2×2 矩陣的反矩陣（伴隨矩陣除以行列式）：

$$\begin{bmatrix} a & b \\ c & d\end{bmatrix}^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a\end{bmatrix}$$

例子：三點 (0, 1)、(1, 3)、(2, 5)。Σx = 3，Σx² = 5，Σy = 9，Σxy = 13，n = 3。行列式 = 5×3 − 3×3 = 6。

$$\alpha = \frac{n\sum xy - \sum x\sum y}{6} = \frac{39-27}{6} = 2,\qquad \beta = \frac{\sum x^2\sum y - \sum x\sum xy}{6} = \frac{45-39}{6} = 1$$

得到 y = 2x + 1，正好穿過三點。

本文用途：Eq.(15)–(17) 正是把這幾個總和逐筆累加的遞迴版本；第 3 部分會證明論文的矩陣 A 其實就是正規方程矩陣的伴隨矩陣。

常見誤解：(1) 「遞迴 OLS 會忘掉舊資料」——不會，本文的版本每個點權重相同，除非重置 (Eq.18)。(2) OLS 是最佳線性不偏估計的前提是誤差同變異 (homoscedasticity)、互不相關；BLE 的重傳尖峰違反這個前提，這正是作者要先做下包絡修正的原因【原文 §III-A 末段】。

1.8 滑動標準差與指數加權平均

直覺：滑動標準差看「最近一段」有多抖；指數加權平均是「新估計 = 舊估計往新觀測靠一點」。

定義：最近 M 筆的平均 μ 與標準差 σ：

$$\mu = \frac{1}{M}\sum_{n=i-M+1}^{i} z_n,\qquad \sigma = \sqrt{\frac{1}{M}\sum_{n=i-M+1}^{i}(z_n-\mu)^2}$$

指數加權平均 (Exponentially Weighted Moving Average, EWMA)：

$$\hat g \leftarrow (1-\lambda)\,\hat g + \lambda\, g_{\text{new}},\qquad 0\le\lambda\le 1$$

λ 越大越相信新觀測、收斂越快、越抖；λ 越小越平滑、越慢。

例子：舊估計 $\hat g$ = 10，新觀測 20，λ = 0.5 → 15；λ = 0.1 → 11。

本文用途：Eq.(7) 是滑動標準差；Eq.(12) 對斜率 γ_0 的更新就是 EWMA，而且 λ 會依延遲抖動程度自動調整 (Eq.10、13)。

常見誤解：EWMA 不是對所有歷史平均；第 k 筆以前的觀測權重是 λ(1−λ)^k，會指數衰減。

1.9 用互相關估計兩訊號的時間差

直覺：把一條訊號左右滑動，看滑到哪個位置跟另一條最吻合。

定義：離散互相關 (Cross-Correlation, CC)

```
R_{xy}[k] = \sum_{n} x[n]\, y[n+k],\qquad \hat\tau = \frac{1}{f_s}\arg\max_k R_{xy}[k]
```

解析度是一個取樣週期 1/f_s。週期訊號（如正弦）每隔一個週期就會再對齊一次，所以可唯一判定的延遲範圍只有 ±T/2。

例子：【原文 §IV-B】訊號以 50 Hz 取樣（解析度 20 ms，太粗），先上取樣到 10 kHz，解析度變 0.1 ms；參考正弦 5 Hz，週期 T = 200 ms，可判定範圍 ±100 ms。作者說同步誤差通常只有數百 μs，遠小於 100 ms，所以不會判錯週期。

本文用途：Eq.(25) 的「真實延遲」就是這樣得到的。

常見誤解：上取樣不會增加資訊。它只是讓 argmax 的格點變細，等同在峰值附近做內插；真正精度還受雜訊與量化影響。

1.10 中位數、四分位距、第 95 百分位數

直覺：中位數是「中間那個人」；四分位距 (Interquartile Range, IQR) 是中間一半人的範圍；第 95 百分位數 (P95) 是「最差的 5% 從這裡開始」。

例子：10 個誤差值 (ms)：0.1, 0.2, 0.2, 0.3, 0.3, 0.4, 0.5, 0.6, 0.9, 3.0。平均 0.65（被 3.0 拉高），中位數 0.35，P95 介於 0.9 與 3.0 之間（numpy 預設線性內插：位置 0.95 × 9 = 8.55，0.9 + 0.55 × 2.1 ≈ 2.06）。

本文用途：Table II 報 20 次重複的中位數與 IQR；Eq.(27) 用每對節點 |RSE| 的 P95 挑出最差的一對。

常見誤解：「中位數 0.6 ms」不代表大部分時間誤差都小於 0.6 ms。這裡的中位數是「20 次實驗的中位數」，每次實驗內的指標又是「最差一對節點在 600 s 內的平均 |RSE|」，是兩層統計。

1.11 學術脈絡：之前的方法與缺點

類別	代表方法（以標題稱）	做法	缺點（作者的評論）
協定無關、重精度	Fine-grained network time synchronization using reference broadcasts（RBS）	接收者比較同一廣播的到達時間	依賴 MAC 層時間戳，非為 BLE 設計
協定無關、重精度	Timing-sync protocol for sensor networks（TPSN）	階層式雙向交換 + MAC 層時間戳	同上
協定無關、重精度	The flooding time synchronization protocol（FTSP）	洪泛傳播 + 線性回歸估偏移與漂移，提出參考點概念	同上
協定無關、重節能	A beaconless asymmetric energy-efficient time synchronization scheme…（BATS）	節點只打時間戳，計算全交給頭節點，sub-μs、省電 95%	仍靠 MAC 層時間戳
BLE 廣播型	CheepSync	收發兩端低層時間戳	只能廣播：單向、不能同時傳設定與資料、較不可靠
BLE 廣播型	BlueSync	非可連線廣播 + 單通道掃描 + Nordic 的 PPI 硬體	同上；多裝置未驗證
BLE 連線型（連線事件）	Synchronization in wireless biomedical-sensor networks with Bluetooth low energy；A study in accuracy of time synchronization of BLE devices using connection-based event	用主從兩端的連線事件對時，差 < 800 μs	不補償漂移，要反覆斷線重連
BLE 連線型（應用層）	Application-layer time synchronization and data alignment method for multichannel biosignal sensors using BLE protocol	時間戳配對 + 線性最小平方	只測 2 個裝置，沒考慮延遲或阻塞的傳輸
BLE 連線型（應用層）	Neural network-based Bluetooth synchronization of multiple wearable devices	神經網路處理時間戳序列，建「虛擬時鐘」	訓練方式不清楚，擴展性不明

類別	代表方法（以標題稱）	做法	缺點（作者的評論）
BLE 連線型（應用層）	Offline synchronization of signals from multiple wireless sensors	離線修正隨機延遲與掉包	只能離線、只測 3 個裝置

【原文 §I-A 末段】SharkTooth 針對的缺口是：**即時、擴展到多於幾個節點、對掉包與重傳穩健、只用標準 BLE 連線模式與應用層**，四者同時成立。

第 2 部分：逐節導讀（與原文平行對照）

每節依序寫：本節要回答的問題、關鍵句解釋、新術語、與前後節的關係、checkpoint。引用原文只取短句，其餘為白話轉述。

§I Introduction（p.46174–46175）

要回答的問題：為什麼體域感測網路 (Wireless Body Sensor Network, WBSN) 需要時間同步？不同步從哪裡來？為什麼選 BLE？

關鍵句與白話：

- 【原文】常用的同步門檻是誤差在一個取樣週期以內，達到「樣本級」精度。白話：sEMG 1 kHz 就要求 < 1 ms；IMU 208 Hz 則是 < 4.8 ms。這個門檻在 §V-C 會再用到。
- 【原文】不同步有兩個來源：本地時鐘頻率不準造成的漂移，以及通訊協定在發送、存取、傳輸、接收各階段加入的延遲。白話：前者是確定性的、可以用直線描述；後者「只要其中一項變得不可預測」就會造成節點間不同步。
- 【原文】商用穿戴系統多用自訂協定，效能好但彼此不相容；標準協定（如 BLE）則能直接連手機、平板、電腦。
- 【原文】BLE 廣播模式快但單向、無安全性；連線模式雙向、有 ACK 與重傳，但重傳帶來可變延遲，對即時同步是挑戰。

新術語：WBSN、ppm、連線間隔 (CI)、GATT、piconet、notification。

與後文關係：這兩個不同步來源直接對應 §III-A Eq.(4) 的 α、β（時鐘）與 ε（協定延遲）。

Checkpoint：你應該能說出「為什麼不能只開機對時一次」（漂移會累積），以及「為什麼連線模式比廣播模式難同步」（重傳造成延遲跳動）。

§I-A Time Synchronization Algorithms（p.46175–46176）

要回答的問題：現有同步方法有哪些？為什麼都不能直接用在「BLE 連線模式 + 即時 + 多節點」？

關鍵句與白話：

- 【原文】協定無關的方法依賴 MAC 層時間戳，而商用 BLE 平台的協定堆疊與韌體通常不開放這層。白話：RBS、TPSN、FTSP 理論漂亮，但手上的 BLE 晶片拿不到需要的時間戳。
- 【原文】BLE 廣播型方法可達數十 μs（CheepSync）甚至數百 ns（BlueSync），但廣播只能單向，不能用 read/write/notify 跟節點互動。
- 【原文】BLE 連線型方法中，有的利用連線事件、誤差 < 800 μs，但不補償漂移、要反覆斷線重連；有的在應用層以封包洪泛加回歸，但只驗證 2–4 個裝置、或只能離線、或訓練細節不明。

新術語：MAC 層時間戳、參考點 (reference point)、洪泛 (flooding)、PPI（Nordic 晶片內的硬體事件連接機制）。

與後文關係：§V-B 的 Table III 會把這些方法並排比較。

Checkpoint：能用一句話說出 SharkTooth 填的缺口（見第 1 部分 1.11 最後一段）。

§I-B Contribution（p.46176）

要回答的問題：作者做了什麼、怎麼驗證？

關鍵句與白話：

- 【原文】作者原本想直接套用文獻方法，但發現那些方法的使用情境假設太簡化，無法用在自家的穿戴系統（即時控制功能性電刺激 (Functional Electrical Stimulation, FES)、sEMG 與運動學擷取）。
- 【原文】步驟：先刻畫 WBSN 的延遲並找出不同步來源 → 設計應用層演算法 → 在多種節點數與封包長度下驗證。
- 【原文】作者強調據其所知，沒有其他同步演算法做過同等規模的驗證。

與後文關係：§II 介紹硬體、§III 方法、§IV 驗證設計、§V 結果。

Checkpoint：知道這篇的主張是「實用、可擴展、易部署」，而不是「精度最高」。

§II System Description（p.46176–46177）

要回答的問題：演算法是在什麼硬體與軟體上開發與測試的？

§II-A 無線感測節點【原文】：

- sEMG 類比前端 (Analog Front-End, AFE)：雙極電極 + 參考電極，增益 ×200–1600，頻寬 30–400 Hz。
- MCU：Apollo3 Blue（ARM Cortex-M4F，SparkFun Artemis 模組），負責 sEMG/IMU 處理、BLE v4.2、USB/I2C。
- IMU：LSM6DSO32，6 軸。
- 外部 32.768 kHz 晶振，±20 ppm，是韌體打時間戳的時間來源。
- PCB 尺寸 33.0 × 18.3 mm（Fig.1）。

§II-B 韌體與 BLE 服務【原文】：

- FreeRTOS 即時作業系統。
- GATT 服務：標準的 Device Information、Battery；自訂的 sEMG 與 IMU 服務，各有 Parameters、Command、Status 特徵值。
- sEMG 1 kHz、14 bit；IMU 16 bit、最高 208 Hz。
- 每 100 ms 發一次 notification；sEMG 封包 205 B、IMU 封包 131 B，都含 4 B 時間戳與 1 B 封包編號。
- 驗證時把 sEMG 通道改成擷取低頻參考電壓，取樣率降到 50 Hz，以便在固定 notification 間隔與 CI 下測不同封包長度。

§II-C 控制平台【原文】：PC 上的 Python 程式；透過 Nordic nRF52840 dongle 當 central，用 Blatann 函式庫控制。作者承認 Python 實作未針對時間關鍵應用最佳化，選擇的是可重現性與易實作。

Checkpoint：能畫出「節點 → dongle → USB → Python」這條路，並指出 t_P 與 t_C 各在哪裡打。

§III SharkTooth Algorithm 開頭（p.46177–46178）

【原文】本節分三段：§III-A 找出影響 notification 時間戳準確度的三個主因；§III-B 同步協定；§III-C 演算法本體。上標 y^i 表示第 i 步，不是次方；次方會寫成 (y^i)^k。

Checkpoint：記住這個上標慣例，否則 Eq.(16) 的 $(t_P^i)^2$ 會讀錯。

§III-A Timestamp Model（p.46178–46179）

要回答的問題：t_C − t_P 由什麼組成？哪些部分可預測、哪些不行？

關鍵句與白話：

- 【原文】目標是把各節點資料表示在共同的取樣時間 t_S 上。
- 【原文】t_P 對節點本地時鐘很精確，但有偏移與漂移；t_C 可以跨節點比較，但含不可預測、隨時間變的延遲。白話：兩個時間戳各有一個缺點，剛好互補。
- 【原文】δT 可拆成漂移係數 α、偏移係數 β、不可預測成分 ε 三部分。
- 【原文】即使沒有重傳，t_P 的產生時刻與連線事件開始時刻不對齊，也會在 ε 裡造成一個 CI 範圍內的不確定性；重傳則讓延遲變成 CI 的倍數。
- 【原文】Fig.3 的最小延遲情境下，δ^2T 的 100 點滑動標準差在 CI 的一半附近擺動，代表無法消除的基礎變異；因為延遲樣式重複且變異近似固定，δT 可以用 OLS 建模。
- 【原文】Fig.4 的雜訊情境下，重傳與軟體延遲打斷週期樣式、產生超過 CI 的尖峰，同變異假設不再成立。

新術語：δT（中心減周邊延遲）、δ^2T（封包間時間，相鄰 δT 之差）、同變異 (homoscedasticity)。

與後文關係：「同變異時可用 OLS、尖峰時不行」是 §III-C 先做延遲修正、再做 OLS 的直接動機；σ_M 會變成 Eq.(10) 的輸入。

Checkpoint：看到 δT 對 t_P 的散點，能指出斜率是 α、截距含 β、上下抖動是 ε、偶爾跳很高的是重傳。

§III-B Synchronization Protocol（p.46179–46180）

要回答的問題：時間戳在實際系統中怎麼產生、怎麼傳？

關鍵句與白話【原文】：

- 不修改標準通訊堆疊、不需要雙向時間戳，只用 BLE notification。
- 控制程式送 serial 指令給 central，central 對周邊做 GATT write 以啟用 notification 並開始擷取。
- 每個樣本在 ADC 產生 data-ready 中斷後立刻打時間戳，存入環形緩衝；湊滿預定樣本數後打包，封包含資料、最後一筆樣本的時間戳 t_P、封包編號。
- BLE 酬載最大 244 B：t_P 4 B + ID 1 B，最多 239 B 給資料（Fig.5）。
- central 收到封包就觸發控制程式的 serial callback，立刻打 t_C；每組 (t_P^i, t_C^i) 即時交給演算法，每個周邊連線獨立執行。
- 作者刻意讓控制程式在 central 外部，central 跑原廠韌體；雖然 serial 層可能加延遲，但目標正是驗證演算法能壓低這些中間層的影響。

Checkpoint：知道 t_P 是「封包內最後一筆樣本」的時間，所以封包內其他樣本的時間要從取樣率往回推。

§III-C Synchronization Algorithm（p.46180–46182）

要回答的問題：如何從含尖峰的 δT 序列即時得到穩定的 α、β？

關鍵句與白話【原文】：

- 既有方法的問題：節點變多 → CI 變大 → δT 變異變大 → 單純回歸收斂慢；而且沒有處理重傳造成的誤差。
- SharkTooth 本質上是線性回歸的穩健版本，用一組策略挑出較不受非確定性離群值影響的資料點。
- 不可預測延遲修正是關鍵創新：輸出 δT* 是一條貼著 δT 分布下界的直線（Fig.6）。
- 每次迭代的 6 個步驟、λ 最大值 0.5 與步長 10^{-6} 是經驗選擇（理由見第 3 部分）。
- 回歸用遞迴 OLS，每步 O(1)，記憶體固定；找到新 pivot 後歸零並等 30 s 再更新。

新術語：pivot 點（δT 的最小值點）、探針序列 (probe) δT_thr、研究係數 (research coefficient) γ_1、穩定期 (settling period)。

與後文關係：§IV 驗證的就是這套流程的輸出 t̂_S。

Checkpoint：能用一句話講 Algorithm 2 在做什麼——「維護一條通過歷史最低點、斜率慢慢調整、始終貼著資料下緣的直線」。

§IV Validation 開頭與 §IV-A Acquisition Set Up（p.46182–46183）

要回答的問題：怎麼設計實驗才能測出不同規模下的同步誤差？

關鍵句與白話【原文】：

- 改變節點數與封包長度，因為這兩者直接影響掉包與重傳機率。
- 刻意用現成元件與 Python，以便重現並涵蓋盡可能多的不確定來源。
- 12 塊節點板由 Arduino UNO R3 提供共同 +5 V（方便自動重複與重啟）。
- 用 RIGOL MSO5104 示波器產生兩路反相、等幅的 5 Hz 正弦，餵進差分放大級，得到 1 Vpp、50 Hz 取樣的訊號。
- 節點距 PC（Intel Core i7 vPro 第 8 代）約 3 m；3 支 nRF52840 dongle、每支 4 個節點、CI 固定 30 ms。
- notification 間隔 100 ms；基本酬載 17 B（資料 + 4 B t_P + 2 B 首末樣本時間差 + 1 B 編號），另外附加 0、110、227 B 的填充。
- 18 種配置（6 種節點數 × 3 種長度），每種 20 次 × 1 小時。

【補充】基本酬載中的資料：50 Hz × 0.1 s = 5 筆 × 2 B = 10 B，10 + 4 + 2 + 1 = 17 B，與原文吻合。三種封包總長 17、127、244 B 對應配置字母 A、B、C（Table I、Table II 明寫）。

Checkpoint：能說出為什麼用「同一個訊號同時餵所有節點」——因為真實的節點間延遲就是 0，任何量到的延遲都是同步誤差。

§IV-B Synchronization Metrics Evaluation（p.46183–46185）

要回答的問題：「同步誤差」具體怎麼算？

關鍵句與白話【原文】：

- 估計延遲：同一個 t_P 樣本在兩個節點的 t̂_S 之差。
- 真實延遲：在未同步的 t_P 時間軸上，用互相關算出各節點訊號相對軟體參考正弦的延遲，兩兩相減。
- 兩組時間戳都先重新取樣到共同、等間隔的 t_P 格點（保持 50 Hz）。
- 以 1 秒為 epoch 平均；上取樣到 10 kHz；參考 5 Hz，週期 200 ms 遠大於典型誤差。
- RSE = 估計延遲 − 真實延遲；每 600 s 一段，每段挑 |RSE| 的 P95 最大的那對節點，算平均 |RSE|、標準差、P95。

Checkpoint：知道 Table II 的每個格子是「20 次實驗 × 每次取最差一對」的中位數與 IQR。

§V Results and Discussion 開頭（p.46185–46186）

要回答的問題：整體表現如何？什麼因素讓誤差變大？

關鍵句與白話【原文】：

- 節點越多、封包越長，RSE 越高；C12 第 1 段中位數 1.27 ms 最高。
- A2 各段中位數都 < 0.5 ms；A2–A12（17 B）各段都 < 1 ms；≤ 4 節點的配置延遲較低。
- 隨時間下降：C12 從 1.27 ms 降到第 6 段 0.62 ms。
- 標準差：C12 第 1 段 1.57 ms；第 2 段起各配置都大幅下降，高節點數配置從第 1 到第 6 段降了一半以上。
- IQR 與中位數同趨勢，表示演算法收斂到實驗內與實驗間都穩定。

Checkpoint：能講出「誤差主要集中在前 10 分鐘」這件事對實際使用的意義（短時間擷取要小心）。

§V-A Correlation Between Synchronization Accuracy and Variance（p.46186–46187）

要回答的問題：能不能在線上就知道同步品質好不好？

關鍵句與白話【原文】：

- P95 的 RSE 與封包間時間的變異 σ^2_100 正相關（Fig.12 的黑色回歸線斜率為正）。
- C12 的 P95 中位數從 3.56 ms 降到 1.28 ms；少於 6 個節點的配置都維持在 1.7 ms 以下。
- 紅色虛線 225 ms^2 ≈ 最佳通訊下可達的最小變異；低於它的點誤差低，高於它的點誤差明顯較高。
- 隨時間推移，變異與誤差的相關變弱，可能是遞迴回歸收斂後對封包時序波動不再敏感。
- 誤差隨節點數上升的可能原因：超過 4 個節點就有兩個以上 piconet 並行，碰撞增加；控制程式的執行緒增多也會讓 t_C 有不可預測延遲。

【補充】225 $ms^2 = (CI/2)^2 = 15^2$。第 3 部分會證明：在 100 ms 送一次、CI = 30 ms 的設定下，δ^2T 的標準差理論值約 14.1 ms，與作者觀察到的「約 CI 的一半」吻合。

Checkpoint：知道 σ^2_100 可以當作「線上同步品質指標」。

§V-B Comparison With the State of the Art（p.46187–46189）

要回答的問題：與既有方法相比如何？

關鍵句與白話【原文】：

- 擴展性：2–12 個節點，既有方法多為 2–4 個；8 個節點的 CheepSync 是廣播型。
- 精度：2 個節點中位數 0.21 ms，最差（12 個）0.62 ms。廣播型方法的超低誤差是在不同條件下取得的。
- 連線型方法中，「Neural network-based Bluetooth synchronization…」4 個裝置約 0.356 ms、「Application-layer time synchronization…」2 個裝置約 0.477 ms，與本文最差情況相當。

- 本文是線上方法；「Offline synchronization…」只能事後處理。
- 本文 CI = 30 ms；他人多用 7.5–15 ms，精度較好但擴展性受限。

Checkpoint：注意比較基準不一致（不同 CI、不同節點數、不同誤差定義），比較只能當參考。

§V-C SharkTooth in Biomedical Applications（p.46189–46190）

要回答的問題：給定一個應用的資料量，能預期多少同步誤差？

關鍵句與白話【原文】：

- 網路總吞吐量越高，P95 的 RSE 越高，近似線性（Fig.13），可據此估算新系統的預期誤差。
- 單通道 sEMG 1 kHz × 16 bit = 2 kB/s；6 個與 12 個感測器分別是 12 與 24 kB/s，接近驗證範圍上限。這時 P95 中位數高於 1 ms 取樣週期，短時間對齊可能受影響；較長的擷取會收斂改善。
- IMU 6 通道 × 208 Hz × 16 bit ≈ 2.5 kB/s；即使多感測器，P95 仍低於 IMU 的取樣週期（約 4.8 ms）。

Checkpoint：能用自己系統的「每節點資料率 × 節點數」去對照 Fig.13，判斷 SharkTooth 夠不夠用。

§VI Conclusion（p.46189–46190）

【原文】重申三個特性：可擴展（12 節點，且許多配置看起來可再擴充）、穩健（減輕重傳與通訊延遲影響）、即時。未來工作：調整 CI 與封包頻率對精度、吞吐量、可靠度的影響；在真實生醫情境（復健穿戴裝置、遠端監測）驗證；低層實作優化。

Checkpoint：整篇讀完應能回答：(1) δT 的三個成分；(2) Algorithm 2 為何貼下緣；(3) 誤差怎麼量；(4) 誤差隨什麼變大、隨時間怎麼變。

第 3 部分（一）數學精講：時間戳模型 Eq.(1)–(7)

共用符號約定：上標 i 是第 i 個封包（不是次方）；每個量都是「每個節點、每個封包一個純量」，整段資料就是長度 N 的序列，例如 t_P ∈ ℝ^N。時間單位一律是秒，數值例子為了好讀改寫成 ms。

Eq.(1) 時鐘模型（§III-A，p.46178）

1. 原式

```
t_S = (\alpha + 1)\, t_P + \beta \tag{1}
```

2. 符號表

符號	意義	形狀／單位	已知或待估
t_P	節點本地時間戳（封包最後一筆樣本）	純量，s	已知（封包內 4 B）
t_S	同一瞬間在共同參考時間軸（中心端）上的時間	純量，s	未知，要估
α	漂移係數 (drift coefficient)	無單位（s/s），常以 ppm 表示	待估
β	偏移係數 (offset coefficient)	s	待估

3. 白話：節點的時鐘跟參考時鐘之間差一個「比例」（走得快或慢）與一個「平移」（開機時間不同）。只要知道這兩個數，就能把任何 t_P 換算到共同時間軸。

4. 推導【補充】：設理想時間為 t。節點時鐘讀數 $t_P = (1+\rho_P)\,t + \theta_P$，中心時鐘讀數 $C_C(t) = (1+\rho_C)\,t + \theta_C$（見第 1 部分 1.1）。

- 步驟 1：由節點式解出 t（移項後除以 $1+\rho_P$）：

$$t = \frac{t_P - \theta_P}{1+\rho_P}$$

- 步驟 2：代入中心時鐘，定義 $t_S := C_C(t)$：

$$t_S = (1+\rho_C)\frac{t_P - \theta_P}{1+\rho_P} + \theta_C = \underbrace{\frac{1+\rho_C}{1+\rho_P}}_{\alpha+1} t_P + \underbrace{\theta_C - \frac{1+\rho_C}{1+\rho_P}\theta_P}_{\beta}$$

- 步驟 3：比對係數得 $\alpha = (\rho_C - \rho_P)/(1+\rho_P) \approx \rho_C - \rho_P$（因 $\rho_P \ll 1$，一階近似），$\beta = \theta_C - (1+\alpha)\,\theta_P$。

所以 α 是兩個時鐘的**相對**頻率誤差，不是節點自己的 ppm。

5. 數值例子：$\rho_C = 0$、$\rho_P = -20$ ppm（節點走慢）、$\theta_P = 5$ s、$\theta_C = 100$ s。

- $\alpha = 20\times10^{-6} / (1 - 20\times10^{-6}) \approx 20.0004$ ppm
- $\beta = 100 - 1.0000200004 \times 5 \approx 94.9999$ s
- 取 $t_P = 3605$ s：$t_S = 1.00002 \times 3605 + 94.9999 \approx 3700.0720$ s
- 驗算：$t = (3605 - 5)/0.99998 \approx 3600.0720$ s，$t_S = t + 100 = 3700.0720$ s ✓

6. 極端情況：$\alpha = 0$、$\beta = 0$ 表示兩時鐘完全一致。α 恆定只在「有限時間內」成立【原文：會隨溫度與老化改變】，所以後面的回歸必須持續更新。

7. 關係與程式：Eq.(1) 是 Eq.(4)、(19)、(20) 的源頭；Eq.(20) 就是把估計值代回 Eq.(1)。

```
tS = (alpha + 1.0) * tP + beta          # Eq.(1)
```

Eq.(2) 中心時間戳的組成（§III-A，p.46178）

1. 原式

```
t_C = t_S + \varepsilon(t_S) \tag{2}
```

2. 符號表

符號	意義	形狀／單位	已知或待估
t_C	PC 收到封包時打的時間戳	純量，s	已知
ε(t_S)	傳輸、韌體、軟體造成的所有延遲，隨時間變化	純量，s，恆為正	不可直接觀測

3. 白話：PC 看到封包的時間 = 樣本真正發生的時間 + 一路上所有的等待。

4. 推導與拆解【補充】：ε 可再拆成

```
\varepsilon = \underbrace{d_{\min}}_{\text{固定最小延遲}} +
\underbrace{w}_{\text{等下一個連線事件}\in[0,\mathrm{CI})} + \underbrace{k\cdot
\mathrm{CI}}_{\text{重傳 }k\text{ 次}} + \underbrace{s}_{\text{USB/OS/Python
抖動}}
```

【原文】作者把 w 與重傳點名為 ε 的主要成分；d_min 與 s 的拆法是我的整理。

5. 數值例子：t_S = 3700.0720 s，d_min = 2 ms，w = 12 ms，無重傳，s = 0 → t_C = 3700.0860 s。

6. 極端情況與直覺檢查：如果 ε 是常數 c，Eq.(4) 會變成 δT = α t_P + (β + c)，也就是說**單向時間戳無法把「偏移 β」與「固定延遲 d_min」分開**。演算法估到的其實是 β + d_min。只要所有節點的 d_min 相同，比較節點之間時這一項會互相抵消；不相同的部分會變成殘餘誤差（第 6 部分）。

7. 程式：ε 不能直接算，只能從 δT 推回去。

Eq.(3)、(4) 可觀測量 δT 與三成分分解（§III-A，p.46178）

1. 原式

```
\delta T = t_C - t_P \tag{3}
```

```
\delta T = \alpha\, t_P + \beta + \varepsilon(t_S) \tag{4}
```

2. 符號表：δT 是中心減周邊延遲 (peripheral-to-central delay)，單位 s，每個封包一個值，**完全可由兩個已知時間戳算出**。

3. 白話：δT 把「我們想要的」（α t_P + β，一條直線）跟「干擾」（ε）加在一起。整個演算法的工作就是從 δT 裡把直線挖出來。

4. 推導：

- 步驟 1：把 Eq.(1) 代入 Eq.(2)：t_C = (α + 1) t_P + β + ε(t_S)。
- 步驟 2：兩邊減 t_P：t_C − t_P = α t_P + t_P − t_P + β + ε(t_S)。
- 步驟 3：左邊依 Eq.(3) 就是 δT，得 Eq.(4)。

【補充】為什麼不直接對 t_C 回歸？因為 t_C ≈ t_P 加上幾十 ms 到幾秒，斜率接近 1。減掉 t_P 後只剩斜率 10^{-6}～10^{-4} 的小直線，數值上比較好處理，也能直接在圖上看出漂移（Fig.3a）。

5. 數值例子：延續上例，α t_P = 20.0004×10^{-6} × 3605 ≈ 0.0721 s，β ≈ 94.9999 s，ε = 0.014 s → δT ≈ 95.0860 s。用 Eq.(3) 驗算：3700.0860 − 3605 = 95.0860 s ✓。

6. 直覺檢查：Fig.3a 的 δT 從約 1.03 s 爬到約 1.05 s，橫跨約 100 s【圖上目測】，斜率約 2×10^{-4} = 200 ppm；Fig.6 從約 0.29 s（t_P ≈ 5 s）到約 0.32 s（t_P ≈ 140 s），斜率也約 200 ppm。【推測】這遠大於節點晶振 ±20 ppm 的規格，代表作者系統中有一個所有節點共有的系統性斜率（可能來自 PC 端時鐘或時間戳換算），論文未說明來源。這一點在第 6 部分很重要。

7. 程式

```
dT = tC - tP                                # Eq.(3), shape (N,)
```

Eq.(5)、(6) 封包間時間 $\delta^2 T$（§III-A，p.46178）

1. 原式

```
\delta^2 T^i = \delta T^i - \delta T^{i-1} \tag{5}
```

```
\delta^2 T^i = \alpha\left(t_P^i - t_P^{i-1}\right) +
\varepsilon\!\left(t_S^i\right) - \varepsilon\!\left(t_S^{i-1}\right) \tag{6}
```

2. 符號表：δ^2T^i 是相鄰封包 δT 的差，單位 s，長度 N − 1 的序列。注意 δ^2 只是記號，不是二次方。

3. 白話：一階差分把 β 消掉、把 α 的影響縮到幾乎為零，剩下的幾乎全是「延遲的變化量」，所以可以拿來衡量通訊品質。

4. 推導：

- 步驟 1：寫出第 i 與第 i−1 個封包的 Eq.(4)：δT^i = α t_P^i + β + ε^i，δT^{i−1} = α t_P^{i−1} + β + ε^{i−1}。
- 步驟 2：相減，β 抵消，得 Eq.(6)。

- 步驟 3：第一項 α(t_P^i − t_P^{i−1}) = α × 0.1 s。即使 α = 200 ppm 也只有 20 μs，比 ε 的變化（數 ms 到數十 ms）小三個數量級，所以 δ²T ≈ ε^i − ε^{i−1}。這就是原文「封包時間戳差近似常數，δ²T 的變異主要反映 ε」的意思。

4′. 補充推導：為什麼 σ 會在 CI 的一半附近？【補充，已用程式驗證】

只看「等下一個連線事件」的 w。封包每 P 秒產生一次，連線事件每 CI 秒一次。令 r = P mod CI（本文 P = 100 ms、CI = 30 ms，r = 10 ms）。

- 步驟 1：第 i 個封包的等待 w_i = (a − t_gen,i) mod CI，其中 t_gen,i = t_gen,0 + iP，所以 w_i = (w_0 − i·r) mod CI。
- 步驟 2：相鄰差只有兩種情況：若 w_{i−1} ⩾ r，沒有繞回，w_i − w_{i−1} = −r；若 w_{i−1} < r，繞回一圈，w_i − w_{i−1} = CI − r。
- 步驟 3：長期下 w 均勻分布在 [0, CI) 時，繞回的比例 = r/CI。
- 步驟 4：平均 = (1 − r/CI)(−r) + (r/CI)(CI − r) = −r + r²/CI + r − r²/CI = 0。
- 步驟 5：變異 = 二階動差 = (1 − r/CI) r² + (r/CI)(CI − r)²。提出 r(CI − r)/CI：

```
\operatorname{Var}(\delta^2 T) = \frac{r^2(\mathrm{CI}-r) + r(\mathrm{CI}-
r)^2}{\mathrm{CI}} = \frac{r(\mathrm{CI}-r)\,[\,r + (\mathrm{CI}-r)\,]}
{\mathrm{CI}} = r(\mathrm{CI}-r)
```

- 步驟 6：r(CI − r) 在 r = CI/2 時最大，最大值 (CI/2)²。所以在沒有重傳與軟體抖動的情況下，σ(δ²T) ⩽ CI/2。

本文數字：r = 10 ms → Var = 10 × 20 = 200 ms²，σ = 14.14 ms。Fig.3c 的 σ_100 在約 14.3–14.7 ms 之間擺動，多出來的部分就是 USB/軟體抖動。Fig.12 的紅色虛線 225 ms² = (30/2)² 正是這個上界。

我用程式驗證了公式（無抖動、30000 個封包）：

P (ms)	r = P mod 30 (ms)	模擬 σ (ms)	公式 √(r(30−r)) (ms)
90	0	0.00	0.00
100	10	14.14	14.14
105	15	15.00	15.00
110	20	14.14	14.14
115	25	11.18	11.18

對照：若 w 是獨立的均勻分布 U[0, CI)，則 Var(w_i − w_{i−1}) = 2 × CI²/12 = CI²/6，σ = CI/√6 ≈ 12.2 ms，同樣小於 CI/2。

5. 數值例子：r = 10 ms 時等待時間會以 20 → 10 → 0 → 20 → 10 → 0 ms 循環，δ²T = −10, −10, +20, −10, −10, +20 ms。這就是 Fig.3b 每約 0.3 s 重複一次的鋸齒。

6. 極端情況：P 是 CI 的整數倍（r = 0）時，w 幾乎不變，δ²T 的基礎變異趨近 0；這看似很好，但第 6 部分會說明它會讓節點偏差無法被消除。

7. 程式

```
d2T = np.diff(dT)                          # Eq.(5), shape (N-1,)
```

Eq.(7) 滑動標準差 σ_M（§III-A，p.46178）

1. 原式

```
\sigma_M^i = \sqrt{\frac{1}{M}\sum_{n=i-M}^{i}\left(\delta^2 T^n - \mu_M^i\right)^2} \tag{7}
```

2. 符號表

符號	意義	形狀／單位	已知或待估
M	視窗長度（封包數）	整數	超參數
μ_M^i	最近 M 個 δ²T 的平均	純量，s	由資料算出
σ_M^i	最近 M 個 δ²T 的標準差	純量，s	由資料算出，餵給 Eq.(10)

3. 白話：最近一段時間延遲「抖得多厲害」。抖得越厲害，越不該相信新進來的點（Eq.10、13）。

4. 推導與細節：這是標準母體標準差公式（除以 M）。

- 【原文】的加總從 n = i−M 到 i，共 M+1 項，卻除以 M。這可能是筆誤（應從 i−M+1 起算），M = 100 時差異約 0.5%，可忽略。
- 【原文】Fig.3c 與 §V-A 都用 M = 100；演算法實際使用的 M **論文未明說**，【推測】也是 100。
- 前 M 個封包還湊不滿視窗時怎麼算：**論文未說明**。我的實作用目前已有的樣本；σ = 0 時令 λ_0 = 1。

5. 數值例子：最近 3 個 δ²T = (−10, −10, +20) ms，μ = 0，σ = √((100 + 100 + 400)/3) = √200 = 14.14 ms。Fig.4c 雜訊情境下 σ_100 在約 30–50 ms 之間。

6. 極端情況：M 太小，σ 跳動大，λ 跟著亂跳；M 太大，對通訊品質變化反應慢。M = 100 對應 10 s。

7. 程式

```
from collections import deque
d2 = deque(maxlen=M)
d2.append(dT_i - dT_prev)              # Eq.(5)
sigma = np.std(d2)                     # Eq.(7)，np.std 預設除以 M
```

第 3 部分（二）數學精講：不可預測延遲修正 Algorithm 2、Eq.(8)–(14)、初始化 Eq.(21)(22)

Algorithm 2 維護兩條直線與一個「樞紐點」。先把狀態變數列清楚，後面每條式子都是在更新它們。

狀態	意義	單位	初始值（Eq.21、22）
(t_P0, δT_0)	樞紐點 (pivot)：目前為止 δT 的最小值位置	s, s	t_P0 = 0，δT_0 ← ∞
γ_0	輸出線 δT_* 的斜率	s/s	0
(t_P1, δT_1)	探針線的起點：最後一個「合格點」	s, s	t_P1 = 0，δT_1 ← ∞
γ_1	探針線 δT_thr 的斜率（研究係數, research coefficient）	s/s	γ_1 ← ∞

Eq.(8) 輸出延遲線 δT_*（p.46180）

1. 原式

```
\delta T_*^i = \delta T_0 + \gamma_0\left(t_P^i - t_{P0}\right) \tag{8}
```

2. 符號：δT_*^i 是第 i 個封包的「修正後延遲」，單位 s，是 Algorithm 2 的輸出、Eq.(16)(17) 的輸入。

3. 白話：一條通過樞紐點、斜率 γ_0 的直線。【原文】它是「與 δT 分布下界相切」的線性趨勢（Fig.6 紅線）。

4. 為什麼要「貼下緣」而不是「穿過中間」【補充】：由 Eq.(2) 的拆解，ε = d_min + w + k·CI + s，而 w、k·CI、s 全都 ≥ 0。所以

```
\delta T^i \;\ge\; \alpha\, t_P^i + (\beta + d_{\min}) \quad \text{等號成立於 } w = 0,\ k = 0,\ s = 0
```

所有點都在「真實直線 + d_min」之上，而且剛好沒有任何額外等待的點會落在這條線上。「與下界相切的直線」就是這條線；反之，對全部點做平均回歸會把重傳尖峰也算進去。

5. 數值：δT_0 = 305 ms、t_P0 = 0、γ_0 = 3925 ppm、t_P = 3 s → δT_* = 305 + 3.925 × 3 = 316.775 ms（見下方追蹤表第 3 列）。

6. 極端：γ_0 = 0 → 水平線，等於永遠用樞紐點的值；只有在剩餘時間很短或 α = 0 時合理。

7. 程式

```
dTs = dT0 + g0 * (tP - tP0)              # Eq.(8)
```

Eq.(9) 探針線 δT_thr（p.46181）

1. 原式

```
\delta T_{thr}^i = \delta T_1 + \gamma_1\left(t_P^i - t_{P1}\right) \tag{9}
```

2. 符號：δT_thr^i 是第 i 個封包時的門檻（s）。低於它的點才是「合格點」，可以拿來更新 γ_0。

3. 白話：一條從上一個合格點出發的判斷線。太高的點（重傳、卡住）被擋掉；如果很久都沒有合格點，線會慢慢抬頭（Eq.14），避免演算法卡死。

4. 關鍵觀察【補充】：合格點更新後（Eq.12），δT_1 = δT_*^i 且 γ_1 = γ_0，所以**探針線一開始和輸出線重合**。剛更新完時，只有落在輸出線下方的點合格。

5. 數值：δT_1 = 316.775 ms、t_P1 = 3 s、γ_1 = 3926 ppm、t_P = 5 s → δT_thr = 316.775 + 3.926 × 2 = 324.627 ms。

7. 程式

```
thr = dT1 + g1 * (tP - tP1)              # Eq.(9)；g1 = inf 時 thr = inf
```

Eq.(10) 變異權重 λ_0（p.46181）

1. 原式

```
\lambda_0^i = \min\!\left(\frac{\mathrm{CI}}{2\,\sigma_M^i},\ 1\right)
\tag{10}
```

2. 符號：CI 是連線間隔（s，已知；本文 0.030），σ_M^i 來自 Eq.(7)，λ_0 ∈ (0, 1] 無單位。

3. 白話：通訊正常時 λ_0 ≈ 1；延遲抖得比「正常」厲害，λ_0 就變小，後面更新斜率時就更保守。

4. 為什麼分母是 2σ（也就是拿 σ 跟 CI/2 比）【推測，依據第 3 部分（一）4′】：只有連線事件等待時，σ(δ²T) = √(r(CI − r)) ≤ CI/2。所以 CI/2 就是「不可避免的基礎抖動」的上界：σ 不超過它時 λ_0 = 1，超過就按比例打折。原文只說「正常操作下接近 1」，沒有解釋來源。

5. 數值：σ = 14.14 ms → 30/28.28 = 1.06 → λ_0 = 1；σ = 40 ms（Fig.4c 的雜訊水準）→ 30/80 = 0.375。

6. 極端：σ → 0 → CI/0 = ∞ → λ_0 = 1；σ → ∞ → λ_0 → 0，線上方的點幾乎不改變斜率。

7. 程式

```
lam0 = 1.0 if sigma == 0 else min(CI / (2 * sigma), 1.0)   # Eq.(10)
```

Eq.(11) 樞紐點識別（p.46181）

1. 原式

```
\delta T^i < \delta T_0 \;\rightarrow\; \begin{cases} \delta T_0 = \delta T^i
\\ \gamma_0 = 0 \\ t_{P0} = t_P^i \\ \gamma_1 \leftarrow \infty \end{cases}
\tag{11}
```

3. 白話：只要看到比目前最小值還小的 δT，就把它當成新的錨點，斜率歸零重來，而且讓下一個點一定合格（γ_1 = ∞ → Eq.9 的門檻 = ∞）。在 Algorithm 1 裡，新樞紐點還會觸發回歸歸零與 30 s 穩定期（Eq.18）。

4. 為什麼是最小值【補充】：由上面的不等式，最小的 δT 最可能是 w ≈ 0、沒重傳、沒卡住的點，是最接近真實直線的錨點。

6. 極端情況：α < 0【推測，以程式模擬驗證】：若趨勢向下（節點比 PC 快），在 Δt = t_P^i − t_P0 後，只要 ε_i < ε_0 + |α|Δt 就會造成新樞紐點。Δt 越大條件越寬鬆，樞紐點會不斷更新，回歸反覆歸零。我的模擬中 α = −12 ppm 的節點在 1 小時內發生 1269 次樞紐點，回歸永遠沒有跑完穩定期。第 6 部分有完整討論與修正法。

7. 程式

```
if dT < dT0:                                # Eq.(11)
    dT0, g0, tP0, g1 = dT, 0.0, tP, np.inf
```

Eq.(12) 合格點更新（p.46181）

1. 原式

```
\delta T^i < \delta T_{thr}^i \;\rightarrow\; \begin{cases} \gamma_0 =
\left(1-\lambda^i\right)\gamma_0 + \lambda^i\,\dfrac{\delta T^i - \delta T_0}
{t_P^i - t_{P0}} \\ \gamma_1 = \gamma_0 \\ \delta T_1 = \delta T_*^i \\
t_{P1} = t_P^i \end{cases} \tag{12}
```

2. 符號：λ^i ∈ (0, 0.5] 來自 Eq.(13)。(δT^i − δT_0)/(t_P^i − t_P0) 是「樞紐點到目前點的弦斜率 (chord slope)」，記為 s_i，單位 s/s。

3. 白話：新斜率 = 舊斜率與弦斜率的加權平均，就是對 s_i 做指數加權移動平均 (EWMA)。然後把探針線重設到輸出線上。

4. 推導一：弦斜率的偏差【補充】

- 步驟 1：代入 Eq.(4)：δT^i − δT_0 = α t_P^i + β + ε_i − α t_P0 − β − ε_0 = α(t_P^i − t_P0) + (ε_i − ε_0)。
- 步驟 2：除以 Δt = t_P^i − t_P0：

```
s_i = \frac{\delta T^i - \delta T_0}{t_P^i - t_{P0}} = \alpha +
\frac{\varepsilon_i - \varepsilon_0}{\Delta t}
```

- 步驟 3：樞紐點的 ε_0 是最小的一類，所以通常 ε_i − ε_0 ⩾ 0，s_i 偏高；但偏差以 1/Δt 衰減。例如 ε_i − ε_0 = 10 ms：Δt = 10 s 時偏 1000 ppm，Δt = 1000 s 時只偏 10 ppm。這解釋了 Fig.6 開頭幾秒的暫態，也解釋了為什麼要等 30 s 再做回歸。
- 步驟 4：當選到的點剛好也在下界（ε_i = ε_0），s_i = α 完全不偏。探針線與 λ 規則都在偏好這種點。

4′. 推導二：EWMA 展開【補充】：連續 k 次合格更新後（中間沒有新樞紐點），反覆代入得

```
\gamma_0^{(k)} = \prod_{j=1}^{k}(1-\lambda_j)\,\gamma_0^{(0)} +
\sum_{j=1}^{k} \lambda_j \prod_{l=j+1}^{k} (1-\lambda_l)\, s_j
```

樞紐點後 γ_0^{(0)} = 0。若 λ_j 都是 0.5，第 j 個弦斜率的權重是 0.5^{k−j+1}：最新的佔 1/2，前一個 1/4，依此類推。權重總和 1 − 0.5^k → 1。

5. 數值追蹤（以下由我的實作跑出，可逐列手算驗證）

玩具資料：為了好算，封包每 1 s 一個（論文是 0.1 s）；真實線 δT = 300 ms + 200 ppm × t_P；額外延遲 ε−d_min = 5, 25, 15, 5, 80, 15, 5, 25 ms（i = 4 的 80 ms 模擬重傳尖峰）；CI = 30 ms。

i	t_P (s)	δT (ms)	分支	λ_0	λ	γ_0 (ppm)	γ_1 (ppm)	δT_* (ms)
0	0	305.00	樞紐點	1.000	-	0	∞	305.000
1	1	325.20	合格（門檻 ∞）	1.000	0.500	10100.00	10100.00	315.100
2	2	315.40	合格	1.000	0.500	7650.00	7650.00	320.300
3	3	305.60	合格（在線下方）	1.000	0.500	3925.00	3925.00	316.775
4	4	380.80	不合格，γ_1 + 10^{-6}	0.432	-	3925.00	3926.00	320.700
5	5	316.00	合格（在線下方）	0.328	0.500	3062.50	3062.50	320.312
6	6	306.20	合格（在線下方）	0.358	0.500	1631.25	1631.25	314.787
7	7	326.40	不合格，γ_1 + 10^{-6}	0.380	-	1631.25	1632.25	316.419

逐列手算幾個關鍵格：

- i = 1：弦斜率 (325.2 − 305)/1 = 20.2 ms/s = 20200 ppm；λ = 0.5 × λ_0 = 0.5（點在線上方）；γ_0 = 0.5 × 0 + 0.5 × 20200 = 10100；δT_* = 305 + 10.1 × 1 = 315.1。
- i = 2：門檻 = 315.1 + 10.1 × 1 = 325.2 > 315.4 → 合格。δ²T = {20.2, −9.8}，σ = 15.0，λ_0 = 30/30 = 1；弦斜率 = 10.4/2 = 5200；γ_0 = 0.5 × 10100 + 0.5 × 5200 = 7650。
- i = 3：305.6 < 上一個 δT_* (320.3) → λ = 0.5；弦斜率 = 0.6/3 = 0.2 ms/s = 200 ppm，**剛好等於真實斜率**，因為這點的額外延遲 5 ms 與樞紐點相同（推導一步驟 4）。γ_0 = 0.5 × 7650 + 0.5 × 200 = 3925。
- i = 4：門檻 = 316.775 + 3.925 = 320.7 < 380.8 → 尖峰被擋掉；γ_1 只加 1 ppm。尖峰讓 σ 變大，λ_0 降到 0.432。
- i = 6：弦斜率又是 200 ppm，γ_0 = 0.5 × 3062.5 + 0.5 × 200 = 1631.25，持續往 200 收斂。

6. 極端：λ = 1 → γ_0 直接等於最新弦斜率，很抖；λ → 0 → γ_0 不動。λ 上限 0.5 是【原文】經驗選擇。

7. 程式

```
elif dT < thr:                              # Eq.(12)
    g0 = (1 - lam) * g0 + lam * (dT - dT0) / (tP - tP0)
    g1, tP1 = g0, tP
    dT1 = dT0 + g0 * (tP - tP0)             # = δT_*^i
```

Eq.(13) 自適應權重 λ（p.46181）

1. 原式

```
\lambda^i = \begin{cases} 0.5\,\lambda_0^i, & \delta T^i \ge \delta T_*^{i-1}
\\ 0.5, & \delta T^i < \delta T_*^{i-1} \end{cases} \tag{13}
```

3. 白話：落在目前輸出線下方的點，代表線還不夠低，用最大權重 0.5 快速拉下來；上方的點可能含額外延遲，權重再乘上 λ_0 打折。

4. 為什麼不對稱【補充】：由 Eq.(8) 的不等式，真實下界以下不會有點。所以「點在線下方」幾乎確定代表線太高；「點在線上方」則可能只是一個有等待的正常點。兩者證據強度不同，所以權重不同。

5. 數值：λ_0 = 0.375：線上方的點 λ = 0.1875，線下方的點 λ = 0.5。

6. 極端：通訊極糟（λ_0 → 0）時，只有線下方的點會改變斜率，線只會往下貼。

7. 程式

```
lam = 0.5 * lam0 if dT >= dTs_prev else 0.5          # Eq.(13)
```

Eq.(14) 探針斜率遞增（p.46181）

1. 原式

```
\delta T^i \ge \delta T_{thr}^i \;\rightarrow\; \gamma_1 = \gamma_1 + 10^{-6}
\tag{14}
```

3. 白話：沒過門檻就把門檻斜率調高一點，下一個點比較容易過。

4. 推導：探針線其實是一條拋物線【補充】

- 步驟 1：上一個合格點在 t_P1，此時 γ_1 = γ_0（Eq.12）。
- 步驟 2：封包間隔 P，經過 Δt = t_P − t_P1 後約有 k = Δt/P 個不合格封包，γ_1 = γ_0 + k × 10^{-6}。
- 步驟 3：門檻高出輸出線的量：

```
\delta T_{thr} - \delta T_* = (\gamma_1 - \gamma_0)\,\Delta t = \frac{\Delta
t}{P}\cdot 10^{-6}\cdot \Delta t = \frac{10^{-6}}{P}\,\Delta t^2
```

- 步驟 4：P = 0.1 s 時 = 10^{-5} × Δt^2（秒）。門檻高出 1 ms 需 Δt = 10 s；10 ms 需約 31.6 s；一個 CI（30 ms）需約 54.8 s。

意義：短時間內只接受很貼線的點，長時間沒新點就逐漸放寬。【原文】作者說 10^{-6} 是配合 10 Hz 封包率經驗決定，且與晶振漂移同數量級。【推測】如果你的封包率不是 10 Hz，要按步驟 3 的 10^{-6}/P 係數重新調整，否則放寬速度會變。

7. 程式

```
else:
    g1 += 1e-6                          # Eq.(14)
```

Eq.(21)、(22) 初始化（§III-C 末，p.46182）

1. 原式

```
\delta T_* \text{ Initialization} \rightarrow \begin{cases} \delta T_0
\leftarrow \infty \\ t_{P0} = 0 \\ \gamma_0 = 0 \end{cases} \tag{21}
```

```
\delta T_{thr} \text{ Initialization} \rightarrow \begin{cases} \delta T_1
\leftarrow \infty \\ t_{P1} = 0 \\ \gamma_1 \leftarrow \infty \end{cases}
\tag{22}
```

3. 白話：δT_0 = ∞ 保證第一個封包一定是樞紐點；γ_1 = ∞（以及δT_1 = ∞）保證下一個非樞紐點一定合格。

6. 實作陷阱【補充】：∞ × 0 = NaN。若 t_P^i = t_P1（例如重複封包），Eq.(9) 會算出 NaN，比較結果永遠是 False。用 numpy 的 inf 時要先去掉重複時間戳。

7. 程式

```
dT0, tP0, g0 = np.inf, 0.0, 0.0         # Eq.(21)
dT1, tP1, g1 = np.inf, 0.0, np.inf       # Eq.(22)
```

第 3 部分（三）數學精講：遞迴 OLS 與同步時間 Eq.(15)–(20)

初始化 Eq.(21)(22) 已在上一節講完，因為它們初始化的是 Algorithm 2 的狀態。

Eq.(15)–(17) 遞迴最小平方（p.46182）

1. 原式

```
\begin{bmatrix}\alpha^i \\ \beta^i\end{bmatrix} = \frac{1}
{\det\!\left(\mathbf{A}^i\right)}\,\mathbf{A}^i\,\mathbf{b}^i, \quad \text{if
} \det\!\left(\mathbf{A}^i\right) \neq 0 \tag{15}
```

```
\mathbf{A}^i = \begin{cases} \begin{bmatrix} 1 & -t_P^i \\ -t_P^i &
\left(t_P^i\right)^2 \end{bmatrix}, & \text{if } \mathbf{A}^{i-1}_{0,0} = 0
\\[2ex] \mathbf{A}^{i-1} + \begin{bmatrix} 1 & -t_P^i \\ -t_P^i &
\left(t_P^i\right)^2 \end{bmatrix}, & \text{if } \mathbf{A}^{i-1}_{0,0} \neq
0 \end{cases} \tag{16}
```

```
\mathbf{b}^i = \begin{cases} \begin{bmatrix} t_P^i\,\delta T_*^i \\ \delta
T_*^i \end{bmatrix}, & \text{if } \mathbf{A}^{i-1}_{0,0} = 0 \\[2ex]
\mathbf{b}^{i-1} + \begin{bmatrix} t_P^i\,\delta T_*^i \\ \delta T_*^i
\end{bmatrix}, & \text{if } \mathbf{A}^{i-1}_{0,0} \geq 0 \end{cases}
\tag{17}
```

2. 符號表

符號	意義	形狀	單位
A^i	累加矩陣（證明見下：正規方程矩陣的伴隨矩陣）	ℝ^{2×2}	左上無單位（點數）、非對角 s、右下 s²
b^i	累加向量	ℝ^{2×1}	上 s²、下 s
A^{i−1}_{0,0}	上一步左上角元素 = 目前累積的點數	純量	無
α^i, β^i	第 i 步的漂移、偏移估計	純量	s/s, s

3. 白話：對「修正後延遲 δT_*」對 t_P 做普通的直線回歸，但不存歷史資料，只存 5 個累加量（n、Σt、Σt²、Σt·y、Σy）。每來一個封包就加一次，再用封閉式算一次解。

4. 完整推導【補充】：論文只給式子，沒證明為什麼這樣就是 OLS。以下補上。

- 步驟 1（模型）：y_i := δT_*^i = α t_i + β + e_i，第 i 列設計向量 x_i = [t_i, 1]ᵀ。
- 步驟 2（正規方程）：由第 1 部分 1.7，Σx_i x_iᵀ [α; β] = Σx_i y_i，也就是

```
\mathbf{N}\begin{bmatrix}\alpha\\ \beta\end{bmatrix} = \mathbf{c},\qquad
\mathbf{N} = \sum_i \begin{bmatrix} t_i^2 & t_i \\ t_i & 1 \end{bmatrix} =
\begin{bmatrix} \sum t_i^2 & \sum t_i \\ \sum t_i & n \end{bmatrix},\qquad
\mathbf{c} = \begin{bmatrix} \sum t_i y_i \\ \sum y_i \end{bmatrix}
```

- 步驟 3（把論文的 **A** 加總起來）：對 Eq.(16) 的每步加項求和：

```
\mathbf{A} = \sum_i \begin{bmatrix} 1 & -t_i \\ -t_i & t_i^2 \end{bmatrix} =
\begin{bmatrix} n & -\sum t_i \\ -\sum t_i & \sum t_i^2 \end{bmatrix}
```

- 步驟 4（認出伴隨矩陣）：2×2 矩陣 [[a, b], [c, d]] 的伴隨矩陣是 [[d, −b], [−c, a]]。套到 **N**：a = Σt²、b = c = Σt、d = n，得 adj(**N**) = [[n, −Σt], [−Σt, Σt²]]，正好就是 **A**。
- 步驟 5（行列式）：det **A** = nΣt² − (Σt)² = det **N**（2×2 時伴隨矩陣行列式等於原行列式）。
- 步驟 6（反矩陣）：**N**⁻¹ = adj(**N**)/det(**N**) = **A**/det(**A**)。
- 步驟 7：**b** 的定義就是 **c**。所以 [α; β] = **N**⁻¹**c** = **Ab**/det(**A**)，即 Eq.(15)。∎

展開成純量（計算 **Ab** 的兩列）：

```
\alpha = \frac{n\sum t y - \sum t \sum y}{n\sum t^2 - \left(\sum t\right)^2}
= \frac{\overline{ty} - \bar t\,\bar y}{\overline{t^2} - \bar t^{\,2}} =
\frac{\operatorname{Cov}(t,y)}{\operatorname{Var}(t)},\qquad \beta =
\frac{\sum t^2\sum y - \sum t\sum t y}{n\sum t^2 - \left(\sum t\right)^2} =
\bar y - \alpha\,\bar t
```

（第二個等號是分子分母同除以 n^2；β 的最後形式可由正規方程第二列 $\alpha\Sigma t + n\beta = \Sigma y$ 直接得到。）

5. 數值例子：三點 (t, y) = (0, 1)、(1, 3)、(2, 5)。

- 逐步累加：$\mathbf{A}$ = [[1,0],[0,0]] + [[1,−1],[−1,1]] + [[1,−2],[−2,4]] = [[3, −3], [−3, 5]]；$\mathbf{b}$ = [0;1] + [3;3] + [10;5] = [13; 9]。
- det $\mathbf{A}$ = 3 × 5 − (−3)(−3) = 6。
- $\mathbf{Ab}$ = [3×13 − 3×9; −3×13 + 5×9] = [12; 6]，除以 6 得 [α; β] = [2; 1]，即 y = 2t + 1 ✓。

我也用 500 個隨機點比對了遞迴式與 np.polyfit，α 與 β 到小數點後 9 位都一致（α = 1.999206960×10^{-4}，β = 0.300086929）。

6. 極端情況與直覺檢查

- 只有 1 個點：det = 1 · t² − t · t = 0，Eq.(15) 不適用。此時 α、β 該用什麼，**論文未說明**；我的實作保留上一個值。
- 2 個點相距 0.1 s：det = $2(t_1^2 + t_2^2) - (t_1 + t_2)^2 = (t_1 - t_2)^2$ = 0.01，很接近 0，解極不穩定。這是穩定期後前幾秒估計會亂跳的數學原因。
- 【原文】Eq.(17) 第二個條件寫「⩾ 0」，與 Eq.(16) 的「≠ 0」不一致。因為 $\mathbf{A}$_{0,0} 是點數，永遠 ⩾ 0，照字面讀兩個條件會在 = 0 時重疊。【推測】應為筆誤，正確是 ≠ 0。
- **浮點數條件**【補充，已驗證】：det = $n\Sigma t^2 - (\Sigma t)^2$ 是兩個大數相減。我測了 36000 個點（1 小時、每 0.1 s）：t_P 從 0 開始時，float32 仍算得對（α = 1.9999×10^{-4}）；但若節點已開機 3 天（t_P ≈ 259200 s），float32 算出 α = -2.15×10^{-4}，連正負號都錯；float64 仍正確。放到 MCU 上要先把 t_P 減掉一個參考值（例如 t_P0）再累加。

6′. 對一條「線」做回歸是什麼意思【推測】：δT_* 本身就是一條線（Eq.8），只是斜率 γ_0 隨時間調整。OLS 把所有歷史 δT_* 等權平均，等於對 γ_0 的整段軌跡做平滑。優點是把 EWMA 的抖動壓下來；缺點是早期過渡期的偏差會一直留在累加量裡，只能靠後續資料慢慢稀釋。這與 Table II 中「誤差隨區段遞減」的趨勢一致。

7. 程式（O(1) 更新）

```
r = np.array([[1.0, -tP], [-tP, tP * tP]])   # Eq.(16) 的加項
v = np.array([tP * dTs, dTs])                # Eq.(17) 的加項
first = (A[0, 0] == 0)                      # 用「更新前」的 A[0,0] 判斷
A = r if first else A + r
b = v if first else b + v
det = A[0, 0] * A[1, 1] - A[0, 1] * A[1, 0]
if det != 0:
    alpha, beta = (A @ b) / det              # Eq.(15)
```

Eq.(18) 新樞紐點後的重置與穩定期（p.46182）

1. 原式

$$t_P^i \le t_{P0} + 30\,\mathrm{s} \;\rightarrow\; \begin{cases} \alpha^i = 0 \\ \beta^i = 0 \\ \mathbf{A}^i \leftarrow \mathbf{0}_{2\times 2} \\ \mathbf{b}^i \leftarrow \mathbf{0}_{2\times 1} \end{cases} \tag{18}$$

2. 符號：30 s 是穩定期 (settling period)，【原文】作者實作的設定。

3. 白話：每次出現新樞紐點，δT_* 都要重新收斂（Fig.6 紅線開頭幾秒的暫態）。所以把回歸清空，等 30 s 後再累加，避免將暫態寫進回歸。

4. 為什麼是 30 s【推測】：由 Eq.(12) 推導一，弦斜率偏差為 (ε_i − ε_0)/Δt。若 ε_i − ε_0 約為一個 CI（30 ms），Δt = 30 s 時偏差約 1000 ppm，再經 EWMA 平滑後變小。這是一個「暫態大致消失」的經驗選擇，論文沒有做敏感度分析。

6. 字面讀法的後果【推測，重要】：照 Eq.(18)，穩定期內 α = β = 0，代入 Eq.(19)(20) 得 t̂_S = t_P，也就是**完全沒有同步**。節點之間的時鐘偏移通常是秒級，這段時間誤差會是秒級。但 Fig.10 第一段的 RSE 大約在 ±2 ms 內。所以作者的程式應該是保留上一組 α、β，或評估時跳過穩定期；論文沒寫是哪一種。我的實作做成可切換的選項（hold_during_settling）。

7. 程式

```
if tP > tP0 + 30.0:   # Algorithm 1, line 5
    ...               # Eq.(15)-(17)
else:                 # Eq.(18)
    A[:] = 0; b[:] = 0
    if not hold:
        alpha, beta = 0.0, 0.0
```

Eq.(19)、(20) 延遲估計與同步時間戳（p.46182）

1. 原式

$$\delta \hat T^i = \alpha^i t_P^i + \beta^i \tag{19}$$

$$\hat t_S^i = t_P^i + \delta \hat T^i \tag{20}$$

2. 符號：δT̂^i 是第 i 個封包的修正量（s），t̂_S^i 是同步後時間戳（s），是整個演算法的最終輸出。

3. 白話：用估計出的直線算「這個封包大約該補多少」，加回本地時間。

4. 推導：Eq.(1) 可改寫成 t_S = t_P + (α t_P + β)。括弧內就是 Eq.(4) 去掉 ε 後的部分，用估計值代入得 Eq.(19)(20)。

【補充】因為 β 的估計含 d_min（Eq.2 的討論），t̂_S 其實估的是 t_S + d_min：節點之間的相對對齊不受影響；但若要跟 PC 上其他來源（例如攝影機、機械臂指令）對絕對時間，會整體偏一個未知的 d_min。

【補充】t_P 只是封包「最後一筆」的時間。封包內第 j 筆（共 n_s 筆、取樣率 f_s）的本地時間是 t_P − (n_s − 1 − j)/f_s，再套同一組 α、β 即可。【原文】驗證用封包另外放了 2 B 的「首末樣本時間差」，可以拿來做這個內插。

5. 數值：α = 200 ppm、β = 0.300 s、t_P = 3600 s → δT̂ = 0.72 + 0.30 = 1.02 s → t̂_S = 3601.02 s。

6. 極端：α = β = 0 → t̂_S = t_P，就是 Eq.(18) 字面讀法的情況。

7. 程式

```
dT_hat = alpha * tP + beta              # Eq.(19)
tS_hat = tP + dT_hat                    # Eq.(20)
# 封包內每筆樣本
tP_samples = tP - (n_s - 1 - np.arange(n_s)) / fs
tS_samples = (1 + alpha) * tP_samples + beta
```

第 3 部分（四）數學精講：評估指標 Eq.(23)–(27)

前置步驟【原文 §IV-B】：第 n 個節點的樣本 s_n^i 有兩組時間戳 t_Pn^i 與 t̂_Sn^i。先把兩組都重新取樣到一組共同、等間距的 t_P^j 格點（所有節點用同一組 t_P^j 數值），取樣率維持 50 Hz。上標 j 是重取樣後的樣本編號，k 是 1 秒的 epoch 編號。

Eq.(23) 估計的節點間延遲（p.46184）

1. 原式

```
\hat\tau_{nm}^j = \hat t_{Sn}^j - \hat t_{Sm}^j = \delta\hat T_n^j -
\delta\hat T_m^j \tag{23}
```

2. 符號：τ̂_nm^j 是第 j 個重取樣點上「節點 n 與 m 的估計時間差」，單位 s；對每一對 (n, m) 都是長度 J 的序列。

3. 白話：兩個節點帶著同一個本地時間數值的樣本，演算法認為它們在真實時間上差多少。

4. 推導：由 Eq.(20)，t̂_Sn^j = t_P^j + δT̂_n^j、t̂_Sm^j = t_P^j + δT̂_m^j。因為兩者用同一個 t_P^j，相減後 t_P^j 抵消，得第二個等號。

5. 數值：δT̂_n = 1.0203 s、δT̂_m = 0.8751 s → τ̂ = 145.2 ms。

6. 直覺：節點開機時間不同，τ̂ 可以是好幾秒；它是「絕對」的時間差，下面 Eq.(25) 的真值則只知道「除以 200 ms 的餘數」。兩者比較前必須處理（見 Eq.26）。

7. 程式

```
grid = np.arange(t_start, t_end, 1 / 50)                  # 共同 t_P^j 格點
dTh = {n: np.interp(grid, tP[n], tS_hat[n] - tP[n]) for n in nodes}
tau_hat = dTh[n] - dTh[m]                                 # Eq.(23)
```

Eq.(24) 一秒 epoch 平均（p.46184）

1. 原式

```
\hat\tau_{nm}^k = \frac{1}{f_s}\sum_{j=k f_s}^{(k+1)f_s - 1}\hat\tau_{nm}^j
\tag{24}
```

2. 符號：f_s = 50 Hz，也就是每個 epoch 有 50 個樣本；τ̂_nm^k 是第 k 秒的平均。

3. 白話：把每秒 50 個估計值平均成一個，讓它跟 Eq.(25) 一秒一個的真值對齊。

4. 檢查：加總範圍從 kf_s 到 (k+1)f_s − 1，共 f_s 項，除以 f_s，所以是算術平均。τ̂ 在一秒內幾乎不變（只有兩節點漂移差，每秒幾十 μs），平均主要是為了時間軸對齊。

5. 數值：一秒內 τ̂ 從 145.19 線性變到 145.21 ms → 平均 145.20 ms。

7. 程式

```
tau_hat_k = tau_hat[: K * 50].reshape(K, 50).mean(axis=1)  # Eq.(24)
```

Eq.(25) 用互相關求「真實」節點間延遲（p.46184–46185）

1. 原式

```
\tau_{nm}^k = \tau_n^k - \tau_m^k \tag{25}
```

2. 符號：τ_n^k 是第 k 個 epoch 裡節點 n 訊號相對軟體參考正弦的延遲（s），在未同步的 t_P 時間軸上計算。

3. 白話與步驟【原文】：

1. 把節點訊號 s_n^i 從 50 Hz 上取樣到 10 kHz，得 s_n^u。
2. 產生軟體 5 Hz 正弦 s̃^u 當共同參考。
3. 切成連續的 1 秒 epoch。
4. 每個 epoch 計算 s_n^u 與 s̃^u 的互相關。
5. 令互相關最大的位移為 τ_n^k。

4. 推導：為什麼相減就是兩節點的延遲【補充】：若節點 n 相對參考延遲 τ_n、節點 m 延遲 τ_m，則 n 相對 m 延遲 τ_n − τ_m，參考訊號抵消。用軟體參考而不是直接互相關 n 與 m，是為了只算 N 次而不是 N(N−1)/2 次。

4′. 解析度與可判定範圍【補充】：解析度 = 1/10 kHz = 0.1 ms。參考是週期 T = 200 ms 的正弦，位移 τ 與 τ + 200 ms 得到同一個互相關最大值，所以 τ 只定義到 ±T/2 = ±100 ms。【原文】作者說預備測試的誤差都在數百 μs，所以 200 ms 週期不會混淆。

5. 數值：τ_n = 37.4 ms、τ_m = −108.2 ms 會被折成 +91.8 ms（−108.2 + 200）→ τ_nm 不是 145.6 而是 −54.4 ms。這就是下一式必須「折回」的原因。

6. 極端：若訊號只有一個 epoch 且雜訊大，互相關峰值會變寬，可能偏一兩個 0.1 ms 格點。【原文】作者用的是 1 Vpp 的乾淨正弦，這個問題很小。

7. 程式

```
from scipy.signal import resample_poly, correlate, correlation_lags
su = resample_poly(s_n_epoch, 200, 1)                       # 50 Hz -> 10 kHz
ref = np.sin(2 * np.pi * 5 * t_u)                           # 軟體參考
c = correlate(su - su.mean(), ref, mode='full')
lags = correlation_lags(len(su), len(ref), mode='full')
tau_n = lags[np.argmax(c)] / 10_000                         # 延遲 [s]，符號慣例需自行確認
```

Eq.(26) 相對同步誤差 RSE（p.46185）

1. 原式

```
\mathrm{RSE}_{nm}^k = \hat\tau_{nm}^k - \tau_{nm}^k \tag{26}
```

2. 符號：RSE (Relative Synchronization Error) 單位 s，每對節點、每秒一個值。

3. 白話：演算法認為的時間差減掉實際的時間差。完美同步時 RSE = 0。

4. 兩個論文沒寫的必要細節【推測】：

- 折回：τ̂ 是絕對值、τ 只知道模 200 ms，所以實際上必須做

```
\mathrm{RSE}_{\text{wrapped}} = \left[\left(\hat\tau - \tau + \tfrac{T}{2}\right) \bmod T\right] - \tfrac{T}{2},\qquad T = 200\ \text{ms}
```

- 符號慣例：互相關的 lag 正負取決於「誰減誰」。要確保 τ̂ 與 τ 同號，否則完美同步時 RSE = 2τ 而不是 0。建議用「人工平移已知量」的合成資料先驗證。

5. 數值：τ̂ = 2345.2 ms（絕對值）、τ = 145.6 ms。折回：(2345.2 − 145.6 + 100) mod 200 − 100 = 2299.6 mod 200 − 100 = 99.6 − 100 = −0.4 ms。

7. 程式

```
T = 0.2
rse = (tau_hat_k - tau_k + T / 2) % T - T / 2            # Eq.(26) + 折回
```

Eq.(27) 每段挑最差的一對（p.46185）

1. 原式

$$\mathrm{RSE}^k_{\max} = \mathrm{RSE}^k_{xy}\quad\text{where}\quad (x,y) = \arg\max_{(n,m)}\left\{\mathrm{P}_{95}\left(\left|\mathrm{RSE}^k_{nm}\right|\right)\right\} \tag{27}$$

2. 符號：P_95 是第 95 百分位數；【原文】每 600 s 一段，一次 1 小時擷取共 6 段。P_95 是對該段內的 600 個 epoch 計算。N 個節點有 N(N−1)/2 對，12 個節點時是 66 對。

3. 白話：每段只看「最差的那一對節點」。之後對這對的 RSE 算三個指標：平均 |RSE|、標準差、P95。每種配置 20 次重複的結果再取中位數與 IQR，就是 Table II。

4. 推導：「最差一對」會隨節點數自動變大【補充＋推測】：假設每個節點有一個獨立的偏差 $b_n \sim \mathcal{N}(0, \sigma^2)$，則一對的誤差是 $b_n - b_m$，最差一對就是 max b − min b（全距）。常態樣本全距的期望值（品管的 d_2 常數）：N = 2 為 1.128σ、N = 4 為 2.059σ、N = 12 為 3.258σ。也就是說**即使每個節點品質完全不變，從 2 個加到 12 個，這個指標就會變成約 2.9 倍**。Table II 第 6 段：A2 → A12 是 0.21 → 0.41 ms（×1.95），C2 → C12 是 0.25 → 0.62 ms（×2.5）。【推測】「節點越多誤差越大」的趨勢，有相當一部分可能來自這個選擇效應，而不全是網路擁塞。作者歸因於 piconet 碰撞與執行緒增加，但沒有分離這兩種效應。

5. 數值：3 個節點、某段的 P95|RSE|：(1,2) = 0.8 ms、(1,3) = 1.1 ms、(2,3) = 0.6 ms → 選 (1,3)，然後拿 (1,3) 的 600 個 RSE 算平均 |RSE|、std、P95。

6. 極端：N = 2 時只有一對，沒有選擇效應。

7. 程式

```
from itertools import combinations
def worst_pair_metrics(RSE, seg):                 # RSE[(n,m)]: shape (K,) 每秒一個
    best = max(combinations(nodes, 2), key=lambda p: np.percentile(np.abs(RSE[p][seg]), 95))
    r = RSE[best][seg]                            # Eq.(27)
    return np.mean(np.abs(r)), np.std(r), np.percentile(np.abs(r), 95)
```

第 4 部分：圖表解讀

從圖上目測的數字標「約」；表格數字直接引用。

Fig.1 系統總覽（p.46177）

- **內容**：左為節點 PCB 正反面（33.0 × 18.3 mm，含 micro-USB、sEMG 前端、處理器與無線模組）；中為節點貼在肩、上臂、前臂的示意；右為 PC 控制平台即時顯示。

- **作者要說的**：這是實際穿戴系統，不是開發板推演。
- **對你的用處**：這個尺寸是「自己畫 MCU + AFE 板」可以達到的參考。

Fig.2 時間戳流程示意（p.46178）

- **內容**：中心先啟動 A 再啟動 B。A 的 t_P = 1, 2 對應 t_C = 1.75, 2.70；B 的 t_P = 1, 2 對應 t_C = 2.25, 3.20（示意單位）。
- **自己算一次**：δT_A = 0.75, 0.70；δT_B = 1.25, 1.20。兩節點的 δT 差 0.5，就是啟動時間差造成的偏移；「同一個 t_P = 1」在兩節點代表不同的真實時間。
- **注意**：純示意圖，數字不是量測值。

Fig.3 最小延遲情境下的 δT（p.46179）

- **座標**：(a)(b) 橫軸 t_P (s)、縱軸 δT (s)；(c) 縱軸 σ_100 (ms)。CI = 30 ms。
- **先看哪裡**：(b) 的鋸齒，約每 0.3 s 重複，範圍約 1.03–1.055 s。
- **作者要證明**：沒有干擾時 δT = 線性趨勢 + 一個 CI 內的週期性波動，變異近似固定，所以可用 OLS。
- **我讀出的**：(1) 鋸齒週期 0.3 s = 3 個封包，與第 3 部分（一）4′「r = 10 ms、三個一循環」的推導吻合。(2) (c) 的 σ_100 約 14.3–14.7 ms，理論值 $\sqrt{200}$ = 14.14 ms，多出的約 0.3 ms 是軟體抖動。(3) (a) 的斜率約 200 ppm，遠大於 ±20 ppm 晶振規格（第 6 部分討論）。
- **要注意**：只展示一個節點、約 100 s。

Fig.4 雜訊情境下的 δT（p.46179）

- **座標**：同 Fig.3。
- **先看哪裡**：(b) 中基礎鋸齒仍在（約 1.19–1.21 s），但間歇跳到約 1.3 s。
- **作者要證明**：重傳與軟體延遲造成尖峰，σ_100 升到約 30–50 ms，同變異假設失效。
- **我讀出的**：尖峰高約 0.1 s，是 CI 的 3 倍以上；單次重傳只加 30 ms，所以這些尖峰可能是連續重傳或 PC 端軟體延遲，論文沒有區分兩者。

Fig.5 同步協定與封包格式（p.46180）

- **內容**：三條時間線（應用程式、中心、周邊）。serial 指令 → GATT write → 開始取樣 → notification（帶 t_P^0, t_P^1）→ callback 打 t_C^0, t_C^1 → SharkTooth → $\hat{t}$_S^0, $\hat{t}$_S^1。下方封包格式：資料最多 239 B、t_P 4 B、ID 1 B。
- **對你的用處**：節點韌體只需要做兩件事：ADC 中斷打時間戳，封包放 t_P 與 ID。

Fig.6 不可預測延遲修正（p.46181）

- **座標**：橫軸 t_P（0–140 s）、縱軸 δT (s)。藍點 = 輸入 δT，紅線 = 輸出 δT_*，綠星 = 樞紐點。

- **先看哪裡**：紅線貼著藍點的下緣。
- **作者要證明**：輸出是與下界相切的線，尖峰不影響它。
- **我讀出的**：(1) 樞紐點在約 $t_P = 5$ s、$\delta T \approx 0.29$ s。紅線在開頭有一段幾乎垂直的上衝（到約 0.41 s），就是 Eq.(12) 推導一中「弦斜率偏差以 $1/\Delta t$ 衰減」的暫態，和第 3 部分玩具追蹤表的前幾列一樣。(2) 下緣從約 0.29 s 升到約 0.32 s，斜率也約 200 ppm。

Fig.7、Fig.8 驗證平台（p.46183–46184）

- **內容**：Fig.7 是照片，Fig.8 是示意圖。A = Arduino 供 +5 V 與 GND；B = 示波器輸出反相的 S1、S2；C = 12 個節點（兩欄、各 6 個）；D = PC 與 3 支 BLE dongle，距離約 3 m。
- **要注意**：節點全部擠在同一塊麵包板、共用電源、靜止不動。穿戴時的人體遮蔽、移動、電池供電都沒有測到（第 6 部分）。

Fig.9 同步前後對照（p.46184）

- **座標**：上圖橫軸 t_P、下圖橫軸 $\tilde{t}_S$，約 2357–2358 s；縱軸電壓 0.5–1.5 V；10 個節點。
- **作者要證明**：同步後波形重合。
- **要注意**：這是一個時間點的視覺示範。以 5 Hz 正弦來看，跟差整數個週期（200 ms 的倍數）的波形也會看起來重合。要看定量結果請看 Table II。

Fig.10 四個節點的 RSE 時序（p.46185）

- **座標**：橫軸輸出時間戳 (s)，0 到約 3700 s；縱軸 RSE (ms)。六條線 = 四個節點的 6 對；黃帶 = ±1 ms；藍虛線 = 每 600 s 分段。
- **先看哪裡**：開頭幾百秒的暫態（最低約 −2.5 ms）。
- **作者要證明**：暫態後收斂到 ±1 ms 內。
- **我讀出、作者沒講的**【推測】：收斂後各對的曲線並不在 0 附近上下對稱抽動。有幾條線長期偏在正側約 +0.3 ms，有一條偏在負側約 −0.3 到 −0.5 ms。代表個別節點有「常數偏差」，不只是隨機抖動。這與下方 Table II 的分析一致。
- **要注意**：只是一次擷取的一組配置，論文沒說是哪一種封包長度。

Table I 驗證參數（p.46182）

參數	數值
配置數	18
節點數	2-4-6-8-10-12
封包長度	17-127-244 B
每次擷取時長	3600 s

參數	數值
每種組合重複	20
連線間隔	30 ms
參考訊號	0.9 + 0.5 sin(10πt) V
取樣率	50 Hz

- 【補充】sin(10πt) = sin(2π · 5 · t)，就是 5 Hz；峰峰值 2 × 0.5 = 1 V，與內文的 1 Vpp 一致。
- 【補充】總實驗量 18 × 20 × 1 h = 360 小時。

Table II RSE 指標（p.46186）：表中數字與我的分析

怎麼讀：列 = 18 種配置（字母 = 封包長度 A17/B127/C244 B，數字 = 節點數）；欄 = 6 個 600 s 區段；格子 = 20 次重複的中位數（IQR），單位 ms。上半部是最差一對的平均 |RSE|，下半部是它的標準差。顏色越深代表數值越大。

關鍵數字（第 1 段 → 第 6 段）：

配置	平均絕對 RSE (ms)	標準差 (ms)
A2	0.30 → 0.21	0.33 → 0.25
A12	0.57 → 0.41	0.59 → 0.25
B2	0.39 → 0.22	0.45 → 0.26
B12	0.82 → 0.50	0.94 → 0.30
C2	0.43 → 0.25	0.55 → 0.29
C12	1.27 → 0.62	1.57 → 0.41

作者要證明：節點越多、封包越長，誤差越大；隨時間收斂；IQR 同趨勢代表實驗間穩定。

我讀出、作者沒講的：

1. **第 2 段以後，標準差幾乎只看封包長度、不看節點數**。A 系列全在 0.24–0.28 ms，B 系列 0.26–0.37 ms，C 系列 0.29–0.54 ms。但平均 |RSE| 明顯隨節點數上升。
2. **用折疊常態分布反推偏差**【推測】：若 RSE ~ 🆖(μ, σ²)，則

$$E|\mathrm{RSE}| = \sigma\sqrt{\tfrac{2}{\pi}}\,e^{-\mu^2/(2\sigma^2)} + \mu\left[1 - 2\Phi\!\left(-\tfrac{\mu}{\sigma}\right)\right]$$

μ = 0 時 E|RSE| = 0.798σ。用第 6 段的兩個數字反解 μ（程式求解）：

配置	平均絕對 RSE	std	若無偏差應為	反推偏差大小
A2	0.21	0.25	0.20	約 0.08 ms
B2	0.22	0.26	0.21	約 0.09 ms
C2	0.25	0.29	0.23	約 0.12 ms
A12	0.41	0.25	0.20	約 0.40 ms
B12	0.50	0.30	0.24	約 0.49 ms
C12	0.62	0.41	0.33	約 0.59 ms

解讀：2 個節點時，RSE 幾乎是零平均的抖動；12 個節點時，最差一對有約 0.4–0.6 ms 的**持續偏差**。這支持兩個解釋（第 6 部分展開）：(a) Eq.(27) 的最差一對選擇效應；(b) 個別節點有常數偏差。注意這是用「兩個中位數」反推，不是同一次實驗的值，只是粗估。

3. **非單調**：A12 第 1 段 0.57 < A10 的 0.61；B12 的 0.82 < B10 的 0.91。代表 20 次重複下仍有不小的隨機性，小差異不要過度解讀。
4. **第 1 段的 IQR 很寬**（C12 0.46、B10 0.55）：短時間使用時的表現不穩定。

要注意：沒有報 P95 的表（只在 Fig.11 中）；沒有報「平均那一對」的誤差，只報最差一對。

Fig.11 RSE 三指標走勢（p.46187）

- **座標**：橫軸區段 1–6；三個子圖縱軸分別是平均 |RSE|、P95 |RSE|、標準差（ms）；顏色 = 節點數，標記形狀 = 封包長度；陰影 = IQR。
- **先看哪裡**：中間的 P95。Table II 沒有這個指標；C12 從約 3.56 ms 降到約 1.28 ms（內文數字）。
- **我讀出的**：標準差子圖在第 2 段就幾乎全部壓到 0.25–0.55 ms；平均與 P95 則要到第 3 段後才平。代表「抖動」很快收斂，「偏差」收斂較慢，符合 OLS 靠累積資料稀釋早期偏差的機制（第 3 部分（三）6′）。

Fig.12 P95 RSE 與封包間時間變異 σ^2_100（p.46188）

- **座標**：六個子圖 = 六段；橫軸 σ^2_100 (ms^2)；縱軸 P95 |RSE| (ms)；每點 = 一種配置在 20 次重複的中位數；黑線 = 線性回歸；紅虛線 = 225 ms^2。
- **作者要證明**：σ^2_100 可當線上品質指標，變異越大誤差越大；後段斜率變小。
- **我讀出的**：(1) 所有點的 σ^2_100 都在約 200 ms^2 以上。理論值 $r(CI - r) = 10 \times 20 = 200\ ms^2$ 正好是「只有連線事件等待」的下限，比作者的 225 ms^2 更精確。(2) 第 1 段斜率最大，約從 (200, 1.0) 到 (870, 3.6)，約 0.004 ms/ms^2。
- **要注意**：只有 18 個點，而且「節點數 × 封包長度」同時驅動變異與誤差（混淆因子）。這是相關，不是「降低變異就能降低誤差」的因果證據。

Table III 與既有方法比較（p.46189）

文獻	年	節點數	模式	CI	平台	線上	用資料封包同步	平均精度
Synchronization in wireless biomedical-sensor networks with Bluetooth low energy [13]	2015	2	連線	未宣告	Bluegiga BLE121	✓	✗	0.03 ms
CheepSync [10]	2016	8	廣播	–	RL78-G14	✓	✗	0.01 ms
A study in accuracy of time synchronization of BLE devices using connection-based event [24]	2017	2	連線	未宣告	Bluegiga BLE121LR	✓	✗	0.75 ms
A synchronized multi-unit wireless platform for long-term activity monitoring [7]	2020	4	廣播	–	nRF52832	✓	✗	0.03 ms
BlueSync [14]	2021	2	廣播	–	nRF52	✓	✗	330 ns per 60 s
Neural network-based Bluetooth synchronization of multiple wearable devices [8]	2023	4	連線	7.5 ms	STM32L476	✓	✗	0.356 ms
Low-power synchronization for multi-IMU WSNs [5]	2023	2	廣播	–	nRF52832	✓	✗	0.2 ms
Application-layer time synchronization and data alignment method for multichannel biosignal sensors using BLE protocol [25]	2023	2	連線	15 / 10 ms	CC2640R2f / nRF52840	✓	✓	0.069 / 0.477 ms
Offline synchronization of signals from multiple wireless sensors [6]	2024	3	連線	未宣告	未宣告	✗	✗	≤ 1 ms
本文	2025	2 → 12	連線	30 ms	Apollo3 Blue	✓	✓	0.21 → 0.62 ms

- **作者要證明**：唯一同時做到連線模式、線上、用資料封包同步、且超過 4 個節點。
- **要注意**：「平均精度」定義不一（有的是漂移率「330 ns per 60 s」，本文是最差一對的平均 |RSE|），CI 與節點數也不同，不是同條件比較。另外 [13] 是連線模式卻有 0.03 ms，比本文好；作者在內文用「不補償漂移、需反覆重連」來區分。

Fig.13 P95 RSE 與網路吞吐量（p.46190）

- **座標**：六個子圖 = 六段；橫軸 = 全網路資料吞吐量 (kB/s)，0–30；縱軸 = P95 |RSE| (ms)；黑線 = 線性回歸。
- **自己驗算橫軸**【補充】：吞吐量 = 節點數 × 封包長度 / 0.1 s。C12 = 12 × 244 / 0.1 = 29.3 kB/s（最右邊的點）；A2 = 2 × 17 / 0.1 = 0.34 kB/s。
- **作者要證明**：可用吞吐量預估誤差，當設計工具。
- **我讀出的**：第 1 段線從約 0.7 ms（0 kB/s）到約 3.3 ms（30 kB/s）；第 6 段從約 0.6 到約 1.3 ms。你的情境：若 4 個節點、每個 2 kB/s（共 8 kB/s），第 6 段約 0.8 ms。
- **要注意**：吞吐量完全由「節點數 × 封包長度」決定，而節點數又決定了對數（66 對 vs 1 對）。外推到 30 kB/s 以上沒有驗證。

第 5 部分：實驗設計與評估

5.1 「資料集」：這篇沒有訓練資料，只有量測設計

項目	內容	來源
訊號	示波器產生的 5 Hz 反相正弦，經差分放大得 1 Vpp，所有節點同時接收	§IV-A
取樣	50 Hz、16 bit（驗證模式下）	§II-B、§IV-A
節點	12 塊自製板，同一塊麵包板、共用 Arduino +5 V	Fig.7、8
中心	3 支 nRF52840 dongle，每支 4 節點；Intel Core i7 vPro 第 8 代 PC；約 3 m	§IV-A
配置	6 種節點數 × 3 種封包長度 = 18 種	Table I
重複	每種 20 次 × 1 小時，共 360 小時	Table I
前處理	重取樣到共同 t_P 格點（50 Hz）→ 1 s epoch → 上取樣 10 kHz → 互相關 → 600 s 分段	§IV-B
執行順序	每種配置自動重複；配置間的執行順序、時間、室內 2.4 GHz 環境	論文未說明
重取樣方法	線性內插或其他	論文未說明

「train/val/test」怎麼對應：演算法沒有可學習的參數，但有 4 個人工調的超參數（λ 上限 0.5、探針增量 10^{-6}、穩定期 30 s、視窗 M）。【原文】作者說前三個是「經驗決定」，**論文未說明**是在哪些資料上調的。

5.2 資料洩漏與循環論證的檢查

檢查項目	狀況	風險
估計值與真值是否獨立	估計只用時間戳；真值用訊號波形的互相關。兩者來源不同	低。這是這個驗證設計最好的地方
超參數是否在測試條件上調	「經驗決定」，資料來源不明	中。若就是在同一套麵包板上調的，結果會偏樂觀
線上或事後	演算法逐封包、因果運算；評估在擷取後做。是否存下線上輸出、還是事後重跑	論文未說明
穩定期的輸出是否納入評估	照 Eq.(18) 字面讀法，穩定期內 t̂_S = t_P	論文未說明。若有剝除，第 1 段的誤差會被低估
重複是否獨立	20 次都是同一組 12 塊板、同一台 PC、同一個場地	硬體變異（不同晶振、不同 dongle）沒有被抽樣

5.3 評估指標

指標	公式	意義
RSE	Eq.(26)：t̂_nm^k − τ_nm^k	一對節點、一秒的同步誤差
最差一對	Eq.(27)：取 P95 絕對 RSE 最大的一對	保守；受對數影響（第 3 部分（四））
平均絕對 RSE	mean(絕對 RSE_max)，每 600 s	同時包含偏差與抖動
標準差	std(RSE_max)	只含抖動，不含偏差
P95	95 百分位絕對 RSE_max	尾端誤差
跨重複統計	20 次的中位數與 IQR	對偶發失敗穩健

【補充】把「平均絕對值」與「標準差」一起看會很有用：兩者接近 0.8 倍關係時，誤差是零平均的抖動；平均絕對值明顯較大時，代表有偏差（第 4 部分 Table II 分析）。

5.4 Baseline 是否公平

- 【原文】作者沒有在自己的平台上重新實作任何既有方法；Table III 的比較是引用各論文自己報的數字。
- 各文獻的 CI（7.5–30 ms）、節點數（2–12）、誤差定義都不同。所以 Table III 只能說明「各方法做過什麼」，不能說明「誰更準」。
- 最自然的 baseline 是「不做延遲修正、直接對 δT 做 OLS」，論文中沒有。

5.5 消融實驗 (ablation)：論文沒做，我用模擬補一個

【原文】論文沒有任何消融實驗，沒有測試拿掉 Algorithm 2、固定 λ、或改變 10^{-6} 與 30 s 的效果。

【推測＋我的模擬】我用自寫的 BLE 時序模型（參數全是我設定的，不是論文數值；4 節點、CI 30 ms、封包 100 ms、1 小時），比較 SharkTooth 與「直接對 δT 做遞迴 OLS」。數字是最差一對的平均絕對 RSE（第 2–6 段，ms）：

情境	直接 OLS	SharkTooth
各節點通道相同、輕度雜訊（1% 尖峰、2% 掉包）	0.26, 0.13, 0.13, 0.15, 0.16	0.32, 0.41, 0.38, 0.35, 0.41
各節點通道相同、重度雜訊（5% 尖峰、10% 掉包）	0.74, 0.97, 0.40, 0.19, 0.24	0.17, 0.19, 0.26, 0.26, 0.20
一個節點掉包 20%、其他 2%	7.24, 7.19, 6.88, 6.85, 6.77	0.69, 0.71, 0.62, 0.66, 0.64

解讀：

- 直接 OLS 估到的是 β + 平均延遲。若所有節點延遲分布相同，平均延遲在「相對」誤差裡會抵消，所以直接 OLS 反而不錯。
- 一旦某個節點掉包率不同（距離、人體遮蔽），直接 OLS 的偏差就是兩節點平均重傳延遲的差。理論值：掉包率 p 時平均重傳次數 = p/(1−p)，30 × (0.2/0.8 − 0.02/0.98) ≈ 6.9 ms，與模擬 6.8–7.2 ms 吻合。SharkTooth 的下緣法不受平均延遲影響，維持在約 0.65 ms。
- 所以「不可預測延遲修正」真正的價值在**各節點延遲統計不同**時。論文的麵包板設置（12 個節點並排、距離相同）剛好最不容易讓這個優勢顯現。
- 這是模擬結果，量值取決於我設的通道參數，只用來說明機制。

5.6 統計顯著性與變異

- 【原文】每種配置 20 次，報中位數與 IQR。這比只報一次好。
- 沒有任何假設檢定（例如配置間差異、區段間差異是否顯著），也沒有信賴區間。Fig.12、13 的回歸線沒有報斜率、R^2 或不確定性。
- 18 個配置點的線性回歸，是對「20 次的中位數」做的，抓不到重複間的變異。
- 硬體樣本量是 12 塊板、一台 PC、三支 dongle；結論外推到其他晶振、其他 BLE 堆疊時要小心。

第 6 部分：批判性閱讀

以下推論凡是引用「模擬」的，都來自我照論文公式寫的實作（第 7 部分）與自訂的通道模型。它們證明「機制存在」，不代表作者的系統一定發生。作者實際的程式可能與論文文字不同。

6.1 隱含假設與實際應用會不會被違反

假設	論文中的依據	實際會不會被違反
A1 δT 的趨勢是「不減」的（α ⩾ 0）	論文沒明說；但 Eq.(11) 用「全域最小值」當錨點只在這個條件下才合理	會。任一節點晶振比 PC 快就 α < 0（見 6.2-1）
A2 下緣會「碰到」真實直線（有些封包的等待 ≈ 0）	「與下界相切」的描述	不一定。取決於封包週期、CI 與相對漂移（見 6.2-2）
A3 所有節點的最小延遲 d_min 相同	沒提	不同 dongle、不同 USB 路徑可能不同。差異會變成永久偏差，單向時間戳偵測不到。論文沒有分開報「同 dongle」與「跨 dongle」的對
A4 漂移在整段時間內幾乎不變	【原文】「有限時間內可視為常數」	穿戴時會。節點從室溫暖到皮膚溫度；【補充】典型 32.768 kHz 音叉晶振溫度係數約 −0.03 ppm/°C²（以元件規格書為準），從 25 °C 到 35 °C 約會改 3 ppm。而遞迴 OLS 沒有遺忘因子，不會追蹤 α 的變化
A5 節點靜止、距離相同、無人體遮蔽	Fig.7、8 的麵包板	穿戴時會。而且這正是 SharkTooth 最有價值的情境（第 5 部分 5.5），但沒有被測到
A6 t_C 在 callback 裡立刻打	【原文】callback 打 t_C 後交給背景執行緒	自己實作時若在佇列之後才打，會把執行緒排程抖動全加進 ε
A7 節點不重開機、不斷線	【原文】用 Arduino 供電是為了失敗時能重啟	會。見 6.2-4

6.2 限制與失敗情境

6.2-1 負漂移會讓樞紐點不斷重置【推測，模擬驗證】

- 推理：由 Eq.(11)，α < 0 時只要 ε_i < ε_0 + |α|Δt 就產生新樞紐點。Δt 越大越容易，而每次新樞紐點都觸發 Eq.(18) 清空回歸並重等 30 s。

- 模擬：4 個節點的α = +15、+5、−3、−12 ppm，1 小時內的樞紐點次數分別是 130、174、846、1269。α < 0 的兩個節點始終等不到回歸開始，α̂ 停在 0，輸出 t̂_S = t_P，誤差是秒級。
- 為什麼作者沒遇到：Fig.3a 與 Fig.6 的斜率都約 +200 ppm，遠大於晶振容差。【推測】他們的系統有一個所有節點共有的正斜率（來源論文未說明），所以 α 全部是正的。
- 修正法（已模擬驗證）：餵給演算法前先加一個已知斜率 B，即 δT′ = δT + B·t_P。取 B = 100 ppm，大於「節點容差 + PC 容差」的上界，保證斜率為正。最後把結果減回：t̂_S = SharkTooth(t_P, t_C + B·t_P) − B·t_P。加一個線性項不改變「哪條線與下界相切」，只改變全域最小值的位置。模擬中樞紐點降到 2–6 次。

6.2-2 封包週期與 CI 的搭配會留下節點常數偏差【推測，模擬驗證】

- 推理：第 3 部分（一）4′ 已推導 w_i = (w_0 − i·r) mod CI。令 g = gcd(P, CI)，則 w 只會落在間距 g 的幾個相位上，最小等待 = (w_0 mod g) ∈ [0, g)。P = 100 ms、CI = 30 ms 時 g = 10 ms。
- 若節點與 dongle 的相對漂移很小，這個最小等待幾乎不變，下緣就永遠比真實直線高 0–10 ms，且每個節點不同。相對漂移夠大時相位會掃過所有位置，下緣每隔 g / |相對漂移| 秒就碰到一次真實直線。
- 模擬（同樣 4 節點、最差一對的平均絕對 RSE，第 2–6 段）：

設定	節點對 dongle 相對漂移	g = gcd(P, CI)	結果 (ms)
P = 100 ms、CI = 30 ms	4–23 ppm（慢）	10 ms	3.42, 4.11, 2.68, 0.89, 2.01
P = 98.75 ms、CI = 30 ms	4–23 ppm（慢）	1.25 ms	0.22, 0.19, 0.20, 0.18, 0.17
P = 100 ms、CI = 30 ms	約 150 ppm（快）	10 ms	0.32, 0.41, 0.38, 0.35, 0.41

- 【補充】BLE 的 CI 以 1.25 ms 為單位。P = 98.75 ms = 79 單位、CI = 30 ms = 24 單位，gcd(79, 24) = 1 單位 = 1.25 ms。**設計規則：讓封包週期與 CI（以 1.25 ms 計）互質**。
- 這也提供了另一個解釋：Table II 的「偏差」與「第一段較大」，可能部分來自相位尚未掃過的節點。

6.2-3 穩定期輸出：照 Eq.(18) 字面實作，每次新樞紐點後有 30 s 輸出 t̂_S = t_P（秒級誤差）。建議保留上一組 α、β，並把第一次穩定期結束前的資料標為「未同步」。

6.2-4 節點重開機或重新連線【推測】：若節點重開機，t_P 從 0 重來，δT = t_C − t_P 會整個往上跳（例如 +1000 s）。這不是新的最小值，不會觸發樞紐點；探針線只以 $10^{-5} \times \Delta t^2$ 的速度上升（Eq.14 推導），要上升 1000 s 需約 10^4 s。所以一定要偵測重開機並重置狀態。論文未說明怎麼處理；封包 ID 只用來排序，演算法本身沒用到。

6.2-5 前 10 分鐘較差：Table II 第 1 段誤差與 IQR 都明顯較大；【原文】P95 在 sEMG 高吞吐配置下會超過 1 ms 取樣週期。短時間的實驗（例如幾分鐘的一個 trial）要留意。

6.2-6 無法估 d_min：t̃_S 與 PC 上其他事件的絕對對齊有一個未知常數偏移（第 3 部分 Eq.2、20）。若你要拿 t̃_S 跟機械臂指令或攝影機對齊，這個偏移要另外用硬體觸發量測。

6.3 審稿人可能提出的質疑與作者可能的回應

1. **「Algorithm 2 到底貢獻多少？」** 沒有「直接 OLS」對照。
   - 可能回應：補一個無修正的 baseline。依我的模擬，在各節點通道不同時差很多（6.8 vs 0.65 ms），在通道相同時差異小。作者很可能需要加一個「不對稱」的實驗才突顯優勢。
2. **「誤差隨節點數上升，是網路擁塞還是取最差一對的選擇效應？」** 1 對 vs 66 對。
   - 可能回應：報「所有對的中位數」或「每個節點對共同參考的誤差」，並分開同 dongle 與跨 dongle 的對。
3. **「超參數（0.5、10^{-6}、30 s、M）的敏感度？是不是在同一套設備上調的？」**
   - 可能回應：做網格掃描；並說明 10^{-6} 與封包率的關係（第 3 部分 Eq.14 推導的 $10^{-6}/P$）。
4. **「麵包板靜態測試能代表穿戴念念？」** 溫度、移動、遮蔽、電池供電都沒測。
   - 可能回應：結論已列為未來工作；可加一組人體穿戴 + 動捕的驗證。
5. **「負漂移、節點重開機、穩定期輸出各怎麼處理？約 200 ppm 的斜率從哪來？」**
   - 可能回應：補充實作細節或公開程式碼。

6.4 重現時可能踩到的坑

坑	論文的狀況	建議
Eq.(7) 視窗 M	未明說（Fig.3 用 100）	用 100；湊不滿時用現有樣本，σ = 0 時 λ_0 = 1
det = 0 時的 α、β	未說明	保留上一個值
Eq.(17) 「≥ 0」	筆誤	用 ≠ 0，而且用更新「前」的 $\mathbf{A}_{0,0}$ 判斷
穩定期輸出	字面為 t̃_S = t_P	保留上一組 α、β
時間單位	10^{-6} 與 30 s 都假設 t 用秒	節點若傳 tick，先換成秒
t_P 的 4 B 繞回	t_P 單位未說明	【補充】若以 µs 計，2^{32} µs ≈ 71.6 分鐘就繞回；以 32768 Hz tick 計約 36.4 小時。要做 unwrap
PC 時鐘來源	未說明	【補充】用單調時鐘（Python 的 time.perf_counter() 或 time.monotonic()）。time.time() 會被系統校時拉動或跳躍
t_C 在哪裡打	callback 內立刻打	不要在佇列後的執行緒裡打

坑	論文的狀況	建議
MCU 上用 float32	未討論	t_P 先減參考值再累加（第 3 部分（三）已驗證）
α < 0	未討論	斜率預偏移 B（6.2-1）
封包週期與 CI	用 100 ms / 30 ms	讓兩者以 1.25 ms 計互質（6.2-2）
互相關的符號與折回	未說明	用已知位移的合成資料先驗證（第 3 部分 Eq.26）

6.5 對你的無線 sEMG + IMU 節點設計的具體意義【推測】

- **精度需求**：路徑 1 用 250 ms RMS 視窗、25 ms 步長。幾 ms 的跨節點誤差對包絡幾乎沒影響；若未來做跨節點的微秒級訊號特徵（例如傳導速度）才需要 sub-ms。
- **節點韌體要做的**：ADC DRDY 中斷打本地時間戳（用硬體計時器）；封包放最後一筆的 t_P + 封包 ID；t_P 用夠寬的單位避免繞回。
- **參數選擇**：每節點資料率、節點數、dongle 數用 §IV-A 的預算法算（第 1 部分 1.4）；封包週期與 CI 互質。
- **演算法**：在 PC 端跑；加斜率預偏移、穩定期保留、重開機偵測。先用第 7 部分的模擬器配你自己的 CI、封包週期跑一次，再上硬體。

第 7 部分：實作對照

以下程式逐行對應論文公式。論文沒有公開程式碼，所以這是依文字重寫的版本。**未完整驗證**：只在合成資料上跑過簡單檢查，不保證與作者的實作一致。

7.1 虛擬碼

```
輸入：每個節點 n 的封包流 (t_P^i, t_C^i)；常數 CI, M=100, λ_max=0.5, step=1e-6, T_settle=30 s
輸出：每個封包的 t̃_S^i

初始化 (Eq.21, 22)：δT0←∞, tP0←0, γ0←0;  δT1←∞, tP1←0, γ1←∞;  A←0, b←0, α←0, β←0

for 每個新封包 (t_P, t_C):
    # ---- Algorithm 2 ----
    δT ← t_C − t_P                                          # Eq.3
    若有上一個 δT：把 δT − δT_prev 放入長度 M 的佇列          # Eq.5
    σ ← std(佇列)                                            # Eq.7
    λ0 ← min(CI/(2σ), 1)（σ=0 時為 1）                       # Eq.10
    if δT < δT0:                                             # Eq.11 樞紐點
        δT0←δT; γ0←0; tP0←t_P; γ1←∞
```

```
    else:
        thr ← δT1 + γ1·(t_P − tP1)                    # Eq.9
        if δT < thr:                                    # Eq.12 合格點
            λ ← λ_max·λ0 if δT ⩾ δT*_prev else λ_max    # Eq.13
            γ0 ← (1−λ)·γ0 + λ·(δT − δT0)/(t_P − tP0)
            γ1 ← γ0; δT1 ← δT0 + γ0·(t_P − tP0); tP1 ← t_P
        else:
            γ1 ← γ1 + step                              # Eq.14
    δT* ← δT0 + γ0·(t_P − tP0);  δT*_prev ← δT*         # Eq.8
    # ---- Algorithm 1 ----
    if t_P > tP0 + T_settle:
        A ← A + [[1, −t_P], [−t_P, t_P²]]               # Eq.16
        b ← b + [t_P·δT*, δT*]                          # Eq.17
        if det(A) ≠ 0: [α, β] ← A·b / det(A)            # Eq.15
    else:
        A←0; b←0; α←0; β←0                              # Eq.18（字面版）
    t̂_S ← t_P + α·t_P + β                               # Eq.19, 20
```

7.2 最小可執行 Python 實作

```
"""SharkTooth (IEEE IoT-J 2025) — 依論文 Algorithm 1 + 2、Eq.(3)-(22) 重寫。
單位一律是秒。每個節點一個物件。未完整驗證。"""
import numpy as np
from collections import deque


class SharkTooth:
    def __init__(self, CI, M=100, lam_max=0.5, gamma_step=1e-6, settle=30.0,
                 hold_during_settling=False):
        self.CI, self.M = CI, M                 # CI: 連線間隔 [s]；M: Eq.(7) 視窗（論文未明說，推測 100）
        self.lam_max, self.step, self.settle = lam_max, gamma_step, settle
        self.hold = hold_during_settling        # False = 照 Eq.(18) 字面：穩定期 α=β=0
        self.dT0, self.tP0, self.g0 = np.inf, 0.0, 0.0      # Eq.(21)
        self.dT1, self.tP1, self.g1 = np.inf, 0.0, np.inf   # Eq.(22)
        self.prev_dT = None
        self.prev_dTs = None
        self.d2 = deque(maxlen=M)
        self.A = np.zeros((2, 2))
        self.b = np.zeros(2)
        self.alpha, self.beta = 0.0, 0.0
        self.n_pivots = 0

    # ---------- Algorithm 2：不可預測延遲修正 ----------
    def _delay_correction(self, tP, tC):
```

```
        dT = tC - tP                                                    # Eq.
(3)
        if self.prev_dT is not None:
            self.d2.append(dT - self.prev_dT)                           # Eq.
(5)
        self.prev_dT = dT
        sigma = float(np.std(self.d2)) if len(self.d2) > 1 else 0.0   # Eq.
(7)
        lam0 = 1.0 if sigma == 0.0 else min(self.CI / (2 * sigma), 1.0)   #
Eq.(10)

        if dT < self.dT0:                                               # Eq.
(11)
            self.dT0, self.g0, self.tP0, self.g1 = dT, 0.0, tP, np.inf
            self.n_pivots += 1
        else:
            thr = self.dT1 + self.g1 * (tP - self.tP1)                  # Eq.
(9)
            if dT < thr:                                                # Eq.
(12)
                lam = self.lam_max * lam0 if dT >= self.prev_dTs else
self.lam_max   # Eq.(13)
                self.g0 = (1 - lam) * self.g0 + lam * (dT - self.dT0) / (tP -
self.tP0)
                self.g1 = self.g0
                self.dT1 = self.dT0 + self.g0 * (tP - self.tP0)        # =
δT_*^i
                self.tP1 = tP
            else:
                self.g1 += self.step                                    # Eq.
(14)
        dTs = self.dT0 + self.g0 * (tP - self.tP0)                      # Eq.
(8)
        self.prev_dTs = dTs
        return dT, dTs

    # ---------- Algorithm 1：一次迭代 ----------
    def update(self, tP, tC):
        dT, dTs = self._delay_correction(tP, tC)
        if tP > self.tP0 + self.settle:                                 #
Alg.1 line 5
            r = np.array([[1.0, -tP], [-tP, tP * tP]])                  # Eq.
(16)
            v = np.array([tP * dTs, dTs])                               # Eq.
(17)
            if self.A[0, 0] == 0:
```

```
                self.A, self.b = r, v
            else:
                self.A, self.b = self.A + r, self.b + v
            det = self.A[0, 0] * self.A[1, 1] - self.A[0, 1] * self.A[1, 0]
            if det != 0:                                                  # Eq.
(15)
                self.alpha, self.beta = (self.A @ self.b) / det
        else:                                                             # Eq.
(18)
            self.A = np.zeros((2, 2)); self.b = np.zeros(2)
            if not self.hold:
                self.alpha, self.beta = 0.0, 0.0
        dT_hat = self.alpha * tP + self.beta                              # Eq.
(19)
        return tP + dT_hat, dT, dTs                                       # Eq.
(20)


class SharkToothBiased:
    """【推測】第 6 部分 6.2-1 的修正：先加斜率 B 讓趨勢為正，再減回。非論文內容。"""
    def __init__(self, CI, B=100e-6, **kw):
        self.B, self.core = B, SharkTooth(CI, hold_during_settling=True,
**kw)

    def update(self, tP, tC):
        tS_hat, dT, dTs = self.core.update(tP, tC + self.B * tP)
        return tS_hat - self.B * tP


if __name__ == "__main__":
    # 用法：每個節點一個物件，每收到一個封包就呼叫一次
    st = SharkTooth(CI=0.030)
    for tP, tC in [(0.0, 0.305), (1.0, 1.3252), (2.0, 2.3154), (3.0,
3.3056)]:
        tS_hat, dT, dTs = st.update(tP, tC)
        print(f"tP={tP:4.1f}  dT={dT*1e3:7.2f} ms  dT*={dTs*1e3:7.3f} ms
tS_hat={tS_hat:.4f}")
```

主程式的輸入就是第 3 部分（二）追蹤表的前 4 列，δT_* 應為 305.000、315.100、320.300、316.775 ms，可用來對照自己的實作。

7.3 關鍵超參數表

參數	論文數值	來源	備註
連線間隔 CI	30 ms	Table I	Eq.(10) 需要它
封包（notification）週期	100 ms	§II-B、§IV-A	影響 Eq.(14) 的有效放寬速度
視窗 M	未說明（Fig.3、§V-A 用 100）	–	推測 100
λ 上限	0.5	Eq.(13)、p.46181	經驗選擇
γ_1 遞增量	10^{-6}	Eq.(14)	配合 10 Hz 封包率
穩定期	30 s	Eq.(18)、Alg.1	經驗選擇
初始化	δT_0 = ∞、γ_0 = 0、δT_1 = ∞、γ_1 = ∞	Eq.(21)(22)	–
穩定期內的輸出	字面為 α = β = 0	Eq.(18)	實際做法未說明
det = 0 時的處理	未說明	–	建議保留上一個值
評估：重取樣率	50 Hz	§IV-B	–
評估：上取樣率	10 kHz	§IV-B	解析度 0.1 ms
評估：參考正弦	5 Hz（0.9 + 0.5 sin(10πt) V）	Table I	可判定 ±100 ms
評估：epoch / 區段	1 s / 600 s	§IV-B	–
評估：尾端指標	P95	Eq.(27)	–
PC 時鐘 API	未說明	–	建議用單調時鐘
t_P 單位（4 B）	未說明	–	注意繞回

第 8 部分：自我測驗

8.1 十題（由淺到深）

Q1（觀念） t_P 與 t_C 各自有什麼優點與缺點？

- **詳解**：t_P 對節點本地時鐘很精確，但帶偏移與漂移，不同節點之間不能直接比較。t_C 在同一個 PC 時鐘上，可以跨節點比較，但含不可預測的傳輸與軟體延遲（§III-A）。SharkTooth 結合兩者的優點。

Q2（觀念） 為什麼不能只在開始時對一次時？

- **詳解**：漂移會累積。±20 ppm 的晶振一小時最多偏 72 ms，兩節點一快一慢可差 144 ms，遠超過 1 ms 的取樣週期。

Q3（計算） 一個 central 接 8 個節點、都送 244 B 封包，CI 至少多少？每節點最大吞吐量多少？

- **詳解**：CI ⩾ 8 × 7.5 = 60 ms；封包週期 ⩾ 2 × CI = 120 ms；吞吐量 ⩽ 244 B / 0.12 s ≈ 2.03 kB/s。剛好夠 1 kHz、16 bit 的單通道 EMG（2 kB/s），沒有餘裕。

Q4（計算） 已知 α = 50 ppm、β = 2.000 s、t_P = 1000 s，求 δT̂ 與 t̂_S。

- **詳解**：δT̂ = 50×10⁻⁶ × 1000 + 2.000 = 2.050 s（Eq.19）；t̂_S = 1000 + 2.050 = 1002.050 s（Eq.20）。

Q5（計算） 封包週期 P = 105 ms、CI = 30 ms、無抖動。δ²T 的標準差是多少？此時 λ_0 是多少？

- **詳解**：r = 105 mod 30 = 15 ms；Var = r(CI − r) = 15 × 15 = 225 ms²；σ = 15 ms（正好是 CI/2 上界）；λ_0 = min(30/(2×15), 1) = 1。

Q6（計算） 用遞迴式 Eq.(15)–(17) 對三點 (1, 2)、(2, 2)、(3, 5) 求 α、β。

- **詳解**：$\mathbf{A}$ = [[3, −6], [−6, 14]]（Σt = 6、Σt² = 1 + 4 + 9 = 14）；$\mathbf{b}$ = [Σty; Σy] = [2 + 4 + 15; 9] = [21; 9]。det = 3 × 14 − 36 = 6。$\mathbf{Ab}$ = [3×21 − 6×9; −6×21 + 14×9] = [63 − 54; −126 + 126] = [9; 0]，除以 6 得 α = 1.5、β = 0。驗算：平均 t = 2、平均 y = 3，β = 3 − 1.5 × 2 = 0 ✓。

Q7（觀念） 為什麼 Eq.(13) 對「在線下方」的點給比較大的權重？

- **詳解**：延遲成分都 ⩾ 0，真實下界以下不會有點。若有點在輸出線下方，幾乎確定是線太高，要快速拉下來。在上方的點可能只是有等待的正常點，證據較弱，權重再乘 λ_0 打折。

Q8（推導） 封包週期 P = 0.2 s 時，探針線要高出輸出線 10 ms 需要多久沒有合格點？

- **詳解**：高度差 = (10⁻⁶/P)Δt² = 5×10⁻⁶ Δt²。令其等於 0.01 s → Δt² = 2000 → Δt ≈ 44.7 s（P = 0.1 s 時是 31.6 s）。封包率變低，放寬就變慢。

Q9（批判） Table II 中 A12 第 6 段平均絕對 RSE = 0.41 ms、std = 0.25 ms。這代表什麼？

- **詳解**：若誤差是零平均常態，平均絕對值應約為 0.798 × 0.25 ≈ 0.20 ms。實際是 0.41，代表最差一對有約 0.4 ms 的持續偏差（粗估，第 4 部分）。可能來源：66 對中挑最差的選擇效應、節點常數偏差、跨 dongle 的 d_min 差異。

Q10（批判） 如果一個節點的晶振比 PC 快 10 ppm（α ≈ −10 ppm），照論文字面實作會發生什麼？

- **詳解**：δT 趨勢向下，全域最小值不斷被刷新，Eq.(11) 反覆觸發新樞紐點，Eq.(18) 反覆清空回歸，可能永遠等不到 30 s 穩定期結束；字面版會輸出 t̂_S = t_P。修正：先加斜率預偏移 B（第 6 部分 6.2-1）。

8.2 「如果改成 X 會怎樣」

E1：如果把 CI 從 30 ms 降到 7.5 ms（只接 1 個節點）？

- 基礎抖動上界 CI/2 從 15 ms 降到 3.75 ms；重傳一次只加 7.5 ms。下緣更清楚，收斂應該更快。【原文 §V-B】他人在 7.5–15 ms CI 的結果較準，但節點數受限。注意 Eq.(10) 的 CI 要跟著改，P 與新 CI 的 gcd 也要重算。

E2：如果把遞迴 OLS 改成「有遺忘因子 μ」的版本（每步 $\mathbf{A} \leftarrow \mu\mathbf{A} + \ldots$、$\mathbf{b} \leftarrow \mu\mathbf{b} + \ldots$）？

- 優點：能追蹤溫度造成的 α 變化；早期暫態偏差會自動被忘掉。缺點：有效記憶長度約 1/(1 − μ) 個封包，μ 太小會變抖。例如 μ = 0.9999 對應約 10^4 個封包 = 1000 s。這是論文沒有的設計。

E3：如果在節點端做同步（例如節點也收到 central 的時間），而不是全部放在 PC？

- 要改成雙向交換（類似 NTP），可以分開偏移與延遲（第 1 部分 1.2），但需要 write/read 來回、增加流量與節點運算。SharkTooth 的取捨是「單向、所有運算在 PC」，節點韌體最簡單。對你自製節點來說，單向版本較容易先做起來。

第 9 部分：術語表

中文	英文	一句話定義	論文首次出現
無線體域感測網路	Wireless Body Sensor Network, WBSN	配戴在身上、以無線連到中心的多個感測節點	Abstract
藍牙低功耗	Bluetooth Low Energy, BLE	低功耗的短距無線標準	Abstract
廣播模式	advertising mode	單向、一對多、不需建立連線的傳輸	Abstract、§I
連線模式	connection mode	central 與 peripheral 建立連線、雙向、有確認與重傳	§I
中心／周邊	central / peripheral	發起連線並排程的一方／被連線的感測節點	§I
微微網	piconet	一個 central 與其幾個 peripheral 組成的網路	§I
連線間隔	Connection Interval, CI	連線模式下資料交換的固定週期，以 1.25 ms 為單位	§I
連線事件	connection event	每個 CI 中實際交換封包的時段	§III-A

中文	英文	一句話定義	論文首次出現
通用屬性設定檔	Generic Attribute Profile, GATT	以 service／characteristic 組織資料並提供 read/write/notify	§I
通知	notification	周邊主動推送資料給中心的 GATT 操作	§II-B
重傳	retransmission	封包未確認時於下一個連線事件重送	§I
百萬分率	parts per million, ppm	10^{-6}，用來表示晶振頻率誤差	§I
時鐘漂移	clock drift	兩時鐘頻率不同造成的時間差累積；本文的 α	§I、Eq.(4)
時鐘偏移	clock offset	兩時鐘開始計時不同造成的常數差；本文的 β	Eq.(4)
周邊時間戳	peripheral timestamp, t_P	節點以本地時鐘對最後一筆樣本打的時間	§III-A
中心時間戳	central timestamp, t_C	PC 收到封包時打的時間	§III-A
同步時間	synchronous time, t_S、$\hat{t}_S$	共同時間軸上的取樣時間及其估計	Eq.(1)、(20)
中心減周邊延遲	peripheral-to-central delay, δT	$t_C - t_P$	Eq.(3)
封包間時間	inter-packet time, $\delta^2 T$	相鄰 δT 之差，用來衡量延遲抖動	Eq.(5)
同變異	homoscedasticity	誤差變異不隨時間或輸入改變的假設	§III-A
普通最小平方法	Ordinary Least Squares, OLS	使殘差平方和最小的線性回歸	§III-A
樞紐點	pivot point	目前 δT 最小值的位置，輸出線的錨點	§III-C、Eq.(11)
修正後延遲	corrected delay, δT_*	貼著 δT 下緣的直線	Eq.(8)
探針序列	probe sequence, δT_{thr}	判斷哪些點可用來更新斜率的動態門檻	Eq.(9)

中文	英文	一句話定義	論文首次出現
研究係數	research coefficient, γ_1	探針線斜率，不合格時每次加 10^{-6}	Eq.(9)、(14)
指數加權移動平均	Exponentially Weighted Moving Average, EWMA	新估計 = (1 − λ) 舊估計 + λ 新觀測；Eq.(12) 的本質（補充術語）	Eq.(12)
穩定期	settling period	新樞紐點後等待 30 s 再做回歸	§III-C、Eq. (18)
伴隨矩陣	adjugate matrix	餘因子矩陣的轉置；本文 $\mathbf{A}$ 即正規矩陣的伴隨矩陣（補充術語）	Eq.(15)
互相關	Cross-Correlation, CC	比對兩訊號在不同位移下的相似度以估計延遲	§IV-B
上取樣	up-sampling	內插成較高取樣率，讓延遲格點變細	§IV-B
相對同步誤差	Relative Synchronization Error, RSE	估計的節點間延遲減真實延遲	Eq.(26)
第 95 百分位數	95th percentile, P95	95% 的值小於等於它	Eq.(27)
四分位距	Interquartile Range, IQR	第 75 與第 25 百分位數之差	§V、Table II
吞吐量	throughput	單位時間傳輸的資料量（kB/s）	§IV-A、Fig.13
商用現成	Commercial Off-The-Shelf, COTS	不需特製的市售元件	Abstract
類比前端	Analog Front-End, AFE	感測訊號放大與濾波電路	§II-A
功能性電刺激	Functional Electrical Stimulation, FES	以電刺激引發肌肉收縮的復健技術	§I-B
參考廣播同步	Reference Broadcast Synchronization, RBS	接收者互比同一廣播的到達時間	§I-A
感測網路定時同步協定	Timing-sync Protocol for Sensor Networks, TPSN	階層式雙向時間戳交換	§I-A
洪泛時間同步協定	Flooding Time Synchronization Protocol, FTSP	洪泛傳播加線性回歸，提出參考點	§I-A

中文	英文	一句話定義	論文首次出現
參考點	reference point	同一瞬間的全域與本地時間戳配對	§I-A